
MOSAIC: Runtime-Efficient Multi-Agent Embodied Planning

Kunjal Panchal^{*1} Saayan Mitra² Sunav Choudhary² Victor Bursztyn² Somdeb Sarkhel² Hui Guan¹

Abstract

LLM-based multi-agent embodied planning remains impractical due to prohibitively high execution latency. We identify failed actions as the dominant bottleneck, stemming from two core challenges: inaccurate state tracking under partial observability and inefficient coordination that produces redundant or conflicting actions. We introduce MOSAIC, a runtime-efficient multi-agent planning framework that addresses both challenges. MOSAIC maintains accurate yet lightweight state tracking through agent-centric semantic memory that stores objects in relative coordinates, enabling geometric transformations and coordination. It ensures efficient coordination through Integer Linear Programming that allocates actions at every planning step, enforcing physical feasibility and inter-agent coordination constraints. Across AI2-THOR and search-and-rescue benchmarks, MOSAIC achieves 27–32% faster execution, 30–33% fewer LLM calls, 25–31% fewer steps, and 4–10% points higher success rates. These results demonstrate that efficient memory and constraint-guided coordination are critical for scalable, low-latency multi-agent planning.

1. Introduction

Many real-world embodied tasks such as collaborative search and rescue, household rearrangement, and environmental exploration require multiple agents (physical or simulated entities executing coordinated plans or policies) operating simultaneously in shared spaces (Liu et al., 2024; Qian et al., 2025; Skrynnik et al., 2024; Chen et al., 2024). Leveraging multiple agents offers clear advantages: they can parallelize subtasks, cover larger areas, and recover from lo-

cal failures (Qian et al., 2025; Nayak et al., 2024), leading to faster and more robust task completion compared to single-agent systems. Recent works have explored the use of Large Language Models (LLMs) as planners in such multi-agent environments, demonstrating impressive inference-time generalization across unstructured tasks and domains (Zhang et al., 2025e; Bai et al., 2025). In this paradigm, LLMs generate action sequences for each agent, while the agents execute these planned actions in the environment.

However, practical deployment of LLM-driven multi-agent planning remains limited due to prohibitively high latency, which include both physical action execution time and LLM inference overhead. For example, running a state-of-the-art multi-agent system (Nayak et al., 2024) to solve a simple embodied task such as “Turn off the faucet and light” requires approximately 7.2 minutes to complete. More complex tasks such as “Rescue two people from a fire-affected area” in a simplified simulation environment require 10.5 minutes to complete.

We find that a substantial portion of the runtime is spent on failed actions during execution, which trigger replanning, recovery, or redundant exploration. Figure 1 illustrates a sequence of such failed actions in a rescue scenario: Agent B repeatedly fails to navigate to a person due to spatial reasoning errors (steps 1–2), then violates action preconditions by attempting to carry the person while holding debris (step 4), while Agent A remains idle waiting for coordination (steps 2–3, 5). In our preliminary experiments on state-of-the-art LLM-based multi-agent systems for embodied planning (Nayak et al., 2024; Zhang et al., 2023b), agents spent up to 16–51% of planning steps on failed actions, frequently attempting unreachable goals due to incorrect spatial estimates or poor coordination. Failed actions increase runtime latency in two critical ways: (a) they consume execution time without contributing to task completion, as agents must retry or replan after each failure; and (b) they cause cascading delays, forcing other agents to stall or execute no-ops while waiting for failed actions to resolve, leading to severe underutilization (as shown in Agent A’s idle periods in Figure 1). As a result, current systems remain impractical for resource-constrained or time-critical applications, requiring up to 9 minutes per episode for tasks such as search and rescue, and 15.6 minutes for fast-paced household activities.

^{*}Partial work completed during an internship at Adobe.

¹College of Information and Computer Sciences, University of Massachusetts, Amherst ²Adobe, San Jose. Correspondence to: Kunjal Panchal <kpanchal@umass.edu>.

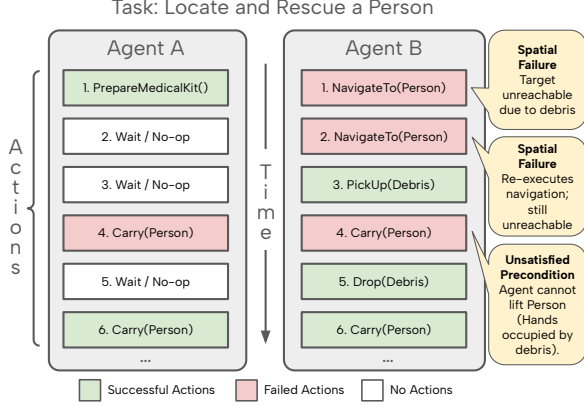


Figure 1. Multi-agent task execution showing failures driven by spatial and coordination inefficiencies. These inefficiencies also induce agent underutilization during stalled periods.

These failures stem from two open challenges in LLM-based multi-agent planning. First, *state tracking under partial observability*: LLMs must maintain accurate beliefs about object locations as agents move and fields of view change, yet without effective tracking mechanisms, agents navigate to outdated locations or re-explore visited areas. Prior work on state tracking faces critical tradeoffs: temporal memories (Nayak et al., 2024; Yao et al., 2022; Lin et al., 2025) are lightweight but force error-prone spatial inference from text, while spatial and spatio-temporal memories (Fang et al., 2019; Hu et al., 2025b; Mao et al., 2025; He et al., 2025) encode geometry explicitly at high computational cost.

Second, *multi-agent coordination*: inadequate task allocation causes redundant behaviors, motion interference, and unmet action preconditions. Existing coordination approaches (Choe et al., 2025; Brawer et al., 2023; Obata et al., 2024; Wu et al., 2024b) perform coarse-grained target-agent allocation or checking for task conflicts through target object dependencies (Choe et al., 2025; Brawer et al., 2023; Obata et al., 2024; Wu et al., 2024b). These assignments assume fixed responsibilities and do not adapt to changing environments or resolve fine-grained, action-level decisions where action failures occur.

In this work, we introduce MOSAIC, a runtime-efficient multi-agent planning framework that addresses both challenges through two novel designs. **(a) Agent-centric Semantic Memory** addresses the state tracking challenge by storing task-relevant objects as structured entries in relative coordinates from each agent’s perspective. Our approach enables geometric transformations through explicit relative positioning while remaining lightweight and scalable: When objects leave an agent’s field of view, their locations are inferred from motion history; cross-agent reasoning is enabled by transforming between agent coordinate frames. **(b) Action Selection via Constraint Optimization** addresses the coordination challenge by selecting one action per agent

from LLM-generated candidates at each timestep via an Integer Linear Programming (ILP) framework. While LLMs can propose contextually plausible actions, they struggle to reason reliably about fine-grained interactions among multiple agents; by formalizing these interactions as symbolic constraints, the ILP efficiently filters infeasible combinations and produces globally consistent action assignments. The constraints capture collision avoidance, blocked-path prevention, and workload balancing, while a cost function penalizes actions with high failure likelihood and workload imbalance. Unlike prior approaches that perform coarse-grained target object allocation over entire episodes, our ILP-based allocation operates at the action level at every planning step, continuously adapting to environmental changes captured by the agent-centric semantic memory.

Empirically, across the AI2-THOR (Kolve et al., 2017) household environment and the SAR (Nayak et al., 2024) search-and-rescue setting, MOSAIC outperforms the strongest baseline in terms of success rate, delivering 27–32% faster execution and 30–33% fewer LLM calls. Under this comparison, agents with MOSAIC execute 25–31% fewer physical steps, driven by a 7–12% point reduction in failed actions. This failure reduction also yields a 4–10% points improvement in task success for MOSAIC under a fixed planning-step budget. Even relative to its single-agent variant, MOSAIC demonstrates substantial gains: by leveraging coordinated action assignments across agents, it reduces redundant exploration and failed attempts, resulting in 1.25–1.33 $\times$ shorter episode duration. The advantages persist at scale: with five agents, we still observe 1.25–1.30 $\times$ end-to-end speedups. Under per-agent sensor noise reflecting real-world conditions, MOSAIC maintains a $\sim 1.25\times$ multi-agent speedup, with task performance remaining within 70–90% of the noiseless setting. Taken together, these findings indicate that MOSAIC delivers runtime efficiency and coordination benefits beyond single-agent deployment, without compromising planning quality.

We summarize the contributions as follows:

1. **Agent-centric Semantic Memory**: A lightweight memory that tracks task-relevant objects and destinations across planning steps, reducing failed actions by up to 12%.
2. **Action selection via constraint optimization**: A constraint optimization ILP framework that converts LLM action proposals into globally consistent multi-agent actions at every planning step. A cost function that reduces unnecessary navigation and high-risk actions, achieving up to 32% faster execution and 33% fewer LLM calls.
3. **Scalable low-latency execution**: End-to-end speedups of up to 1.30 $\times$ with five agents, while improving task success by up to 10%.

2. Related Work

We review prior work on memory representation for embodied planning and agent coordination. Further discussion is deferred to Appendix A.

Memory Representations for Embodied Agents. Work on memory for embodied agents can be categorized into (a) temporal, (b) spatial, and (c) spatio-temporal memory. *Temporal memory* retains past observations without explicitly storing geometry, including short-term memory over limited context windows (Yao et al., 2022; Lin et al., 2025), long-term episodic memory across extended horizons (Nayak et al., 2024; Fang et al., 2019; Gupta et al., 2025a; Lei et al., 2025a; Zhang et al., 2025b; Yadla, 2025), and hierarchical memory organizing information at multiple abstraction levels (Wang et al., 2025; Hu et al., 2025a; Lei et al., 2025b; Zhang et al., 2025a; Han et al., 2025b). While effective for retaining history, temporal memories often rely on unstructured textual logs (Cemri et al., 2026; Zhang et al., 2025d; Li et al., 2024), resulting in error-prone spatial inference from text. *Spatial memory* explicitly stores environmental layout independent of time, including topology-based navigation (Gupta et al., 2017; Chaplot et al., 2020) and dense 3D representations such as VLMAPS (Huang et al., 2023), CONCEPTGRAPHS (Gu et al., 2024), MEM4NAV (He et al., 2025), TME (Ye, 2025), and MEM2EGO (Zhang et al., 2025c), which incur significant overhead for high-fidelity geometry. *Spatio-temporal memory* combines both dimensions, maintaining spatial representations that evolve over time (Fang et al., 2019; Yang et al., 2025; Lei et al., 2025b; Mao et al., 2025; Anwar et al., 2025; Hu et al., 2025b), typically requiring accumulation over long horizons with high computational cost. In contrast, our agent-centric semantic memory maintains a structured textual representation of task-relevant objects in relative coordinates, updated at every planning step to track distances and goals even when objects fall outside the field of view, enabling efficient spatial grounding without dense reconstruction overhead.

Multi-Agent Coordination. Coordination methods differ in how task/action allocation is produced and failures are handled. *Neural methods* propose candidate actions from learned experience but are largely reactive in failure handling, correcting errors only after execution (Yao et al., 2022; Kannan et al., 2024; Zhang et al., 2023b; Wang et al., 2023; Ahn et al., 2022; Ji et al., 2024; Huang et al., 2022). *Symbolic approaches* leverage preconditions and logical rules to proactively reason about feasibility (Zheng et al., 2024; Cornelio & Diab, 2024), commonly through Task and Motion Planning (TAMP) frameworks (Dantam et al., 2018; Kaelbling & Lozano-Pérez, 2011; Lozano-Pérez & Kaelbling, 2014; Garrett et al., 2021; Faroni et al., 2024), but lack the flexible reasoning capabilities of LLMs. *Neuro-symbolic*

coordination uses LLMs to propose actions and symbolic optimization to enforce action feasibility. Choe et al. (2025) uses symbolic constraints to manage failures reactively via help request actions, whereas MOSAIC proactively coordinates all possible actions. Brawer et al. (2023) proposes LLMs to convert user-defined constraints into ILP feasibility checks, but does not address runtime optimization. LIP-LLM (Obata et al., 2024) and Wu et al. (2024b) assign tasks to agents for the entire episode, without reasoning about action-level conflicts at each step. In contrast, MOSAIC performs joint action-level assignments at every timestep, avoiding failures through fine-grained, LLM-guided feasibility reasoning.

3. Methodology

We first formulate the problem setting and introduce a general multi-agent planning architecture to contextualize MOSAIC, and then elaborate on MOSAIC’s two key designs.

3.1. Preliminaries

Problem Setting: Multi-Agent POMDP. The multi-agent embodied planning problem is typically formulated as a Partially Observable Markov Decision Process (POMDP) without access to explicit transition dynamics or reward functions. The environment supports a joint action space $\mathcal{X} = \mathcal{X}_1 \times \dots \times \mathcal{X}_N$, where $\mathcal{X}_i$ denotes the discrete action space of agent a_i . The goal in this work is to satisfy a high-level natural language instruction l by producing a sequence of joint actions $\mathbf{x}_t = \{x_{1,t}, \dots, x_{N,t}\}$, $x_{i,t} \in \mathcal{X}_i$, whose execution leads to fulfilling the semantic requirements of l , while minimizing runtime overhead.

Partial observability implies that agents operate without complete state information and act based on local observations accumulated over time, including past actions, execution outcomes, and internal memory. In this setting, effective action planning requires (a) maintaining and updating internal state representations (analogous to a belief state) rather than relying solely on instantaneous observations, as agents must reason about objects and locations beyond their current field of view, and (b) selecting joint actions via a constrained policy (conditioned on the state and the task), avoiding conflicts under physical constraints, satisfying action preconditions, and preventing redundant exploration (implicitly accounting for transition uncertainty).

Our framework captures the functional roles of a POMDP solver: it maintains an internal state representation that aggregates partial observations (analogous to a belief state) and selects joint actions via a constrained policy conditioned on this state and the task. Enforcing feasibility under physical and resource constraints implicitly accounts for transition uncertainty, approximating the role of a transition model.

Multi-Agent Planning Architecture. MOSAIC is designed to be compatible with general LLM-based planning frameworks. Here, we describe its instantiation within a state-of-the-art centralized planning architecture. This architecture uses distinct LLM calls for: (a) decomposing high-level task instructions into subtasks, (b) proposing actions for each subtask, and (c) verifying subtask completion. This role separation, adopted from prior LLM-based embodied planning systems (Nayak et al., 2024; Zhang et al., 2023b; Kannan et al., 2024), decouples high-level reasoning from low-level action selection to improve execution reliability. Appendix B provides a procedural description, while here we summarize the core conceptual steps.

- (a) High-level Task Decomposition: Given an instruction I and shared memory M , the LLM-PLANNER decomposes the task into a set of open subtasks. The framework maintains an interaction history H and a set of completed subtasks S_c , which together provide persistent context. Execution proceeds iteratively until all subtasks are completed.
- (b) Action Proposals: At each planning step, a centralized LLM-ACTOR proposes joint actions for all agents based on the currently open subtasks. Actions are executed in parallel, producing observations that update H and M .
- (c) Verification of Subtask Completion: After execution, LLM-VERIFIER determines which subtasks have been completed. This step accounts for execution errors and newly revealed dependencies. Then LLM-PLANNER is re-invoked to revise remaining subtasks using the updated context.

The above LLM-based multi-agent planning framework exhibits two key limitations that lead to long time to task completion. First, inefficient state tracking under partial observability degrades the quality of action generation. Second, insufficient coordination of generated actions across agents results in frequent execution failures. We address these issues with MOSAIC, a runtime-efficient planning system composed of two novel designs (Figure 2). First, MOSAIC maintains an **agent-centric semantic memory** (ASM) that tracks task-relevant objects and destinations for each agent (§ 3.2). Second, it introduces action-level coordination through an **integer linear programming (ILP)** framework. This framework resolves LLM-ACTOR action proposals into globally consistent, feasible, and efficient assignments via action constraints and a cost function (§ 3.3).

3.2. Agent-centric Semantic Memory

Agent-centric Semantic Memory (ASM) maintains task-relevant objects and destinations using *agent-centric relative distances*, updated at each planning step based on observations and agent motion. The design is guided by two principles: (a) Lightweight spatial reasoning: tracking actionable spatial relations in relative coordinates is more computationally efficient than constructing global maps or dense

3D representations. (b) Cross-agent transferability: The relative coordinate representation can be translated across agents’ perspectives through simple geometric transformations. Once an agent moves or interacts with an object, others can update their relative distances from that object accordingly. This transfer avoids repeated exploration attempts, motion collisions, and interference.

For each agent ($a \in A$) and task-relevant object or destination ($o \in O$), we maintain agent-centric distance and orientation: $r_{a,o} = \frac{|x_o - x_a| + |z_o - z_a|}{u}$, and $\phi_{a,o} = \text{atan2}(x_o - x_a, z_o - z_a) - \theta_a$, where (x_a, z_a) and θ_a denote the agent’s location and orientation, (x_o, z_o) the object or destination’s global position, and u a unit of motion corresponding to one discrete step. We note that even for 3D simulators, locomotion is effectively planar in (x, z) ; following prior work, spatial failures arise only from horizontal movement, as vertical actions do not fail and crouch/stand is not required. These coordinates are continuously updated as the agent moves. I.e., Forward motion decreases the relative distance ($r_{a,o}^{\text{new}} = r_{a,o}^{\text{old}} - d$), and changes in heading adjust the relative orientation ($\phi_{a,o}^{\text{new}} = (\phi_{a,o}^{\text{old}} - \Delta\theta_a) \bmod 360^\circ$). Lateral movements are updated similarly. Additionally, re-observations implicitly relocalize objects by updating their coordinates, correcting accumulated sensor noise.

To encode spatial information semantically, distances $r_{a,o}$ are converted into semantic tiers: an object 1–2 steps ahead is labeled ‘just ahead’, while >10 steps away is labeled ‘far’. Orientations $\phi_{a,o}$ are similarly mapped to semantic directions, e.g., ‘left’, ‘right’, or ‘ahead’. These underlying continuous metrics are only discretized within the LLM-ACTOR prompt to facilitate reasoning, preventing the accumulation of discretization errors over long horizons. This discretization is applied only at the interface level, while the underlying memory retains full continuous geometry, ensuring numerical precision while providing a stable and compact interface for LLM reasoning. Figure 2 illustrates these semantic tiers in the Agent-centric Semantic Memory: Agent A sees the Bowl ‘just ahead (2 moves)’ while Agent B perceives it ‘far left (12 moves)’.

The memory also stores relative distances between agents, enabling cross-agent transferability of information. Let $(r_{a,b}, \phi_{a,b})$ be the relative distance and orientation between agents a and b , and $(r_{a,o}, \phi_{a,o})$ between agent a and object o . We define the relative displacement in global coordinates:

$$\begin{aligned}\Delta x &= r_{a,o} \sin(\phi_{a,o} + \theta_a) - r_{a,b} \sin(\phi_{a,b} + \theta_a), \\ \Delta z &= r_{a,o} \cos(\phi_{a,o} + \theta_a) - r_{a,b} \cos(\phi_{a,b} + \theta_a).\end{aligned}$$

Then, agent b ’s location and orientation with respect to the object o would be:

$$r_{b,o} = \frac{|\Delta x| + |\Delta z|}{u}, \text{ and } \phi_{b,o} = \text{atan2}(\Delta x, \Delta z) - \theta_b.$$

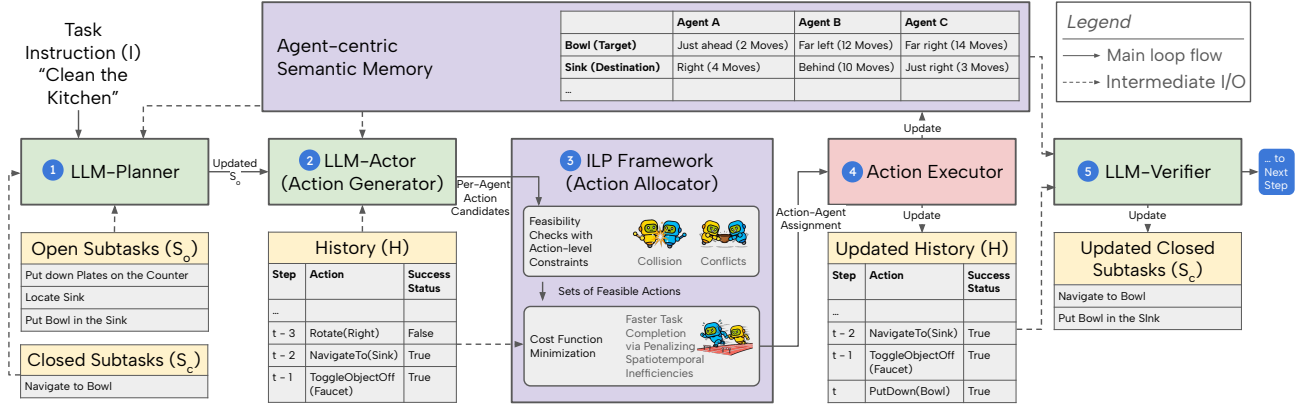


Figure 2. Overview of MOSAIC. **Agent-centric semantic memory** (ASM) maintains task-relevant spatial context across timesteps under partial observability, improving action generation. The LLM-ACTOR proposes action candidates based on ASM, which an **integer linear programming (ILP)** framework resolves into globally consistent joint actions through **feasibility constraints**. A **cost function** guides the ILP to balance agent workloads and reduce navigation and temporal overhead. Together, these components reduce execution failures and redundant actions, enabling low-latency, fine-grained multi-agent coordination.

3.3. Action Selection via Constraint Optimization

Next, MOSAIC introduces an Integer Linear Programming (ILP) framework that selects one action per agent from LLM-ACTOR-generated candidates. It addresses a limitation of prior works: while LLMs excel at proposing contextually relevant actions, they cannot guarantee constraint satisfaction across multiple agents. Conversely, constraint optimization methods can enforce feasibility guarantees but lack the semantic understanding to generate meaningful action proposals. MOSAIC combines both strengths, leveraging LLMs for candidate action generation and ILP for constraint enforcement.

Action-Level Feasibility Constraints. At each planning step, the LLM-ACTOR generates a small set of plausible action candidates for each agent $a_i \in A$: $X_{i,t} = \{x_{i,t}^{(1)}, x_{i,t}^{(2)}, \dots\}$. The LLM-ACTOR generates action candidates *per agent*, respecting each agent’s local context to avoid irrelevant proposals. It produces *multiple candidates* per agent, allowing the ILP framework to resolve conflicts and trade-offs and select a feasible and conflict-free joint action. The ILP framework selects exactly one action per agent at each planning step through binary variables

$$z_{i,t}^{(k)} = \begin{cases} 1 & \text{if agent } a_i \text{ executes candidate } x_{i,t}^{(k)} \text{ at step } t, \\ 0 & \text{otherwise.} \end{cases}$$

The executed action for agent a_i at planning step t is given by $x_{i,t} = \sum_k z_{i,t}^{(k)} x_{i,t}^{(k)}$, with constraints enforcing $\sum_k z_{i,t}^{(k)} = 1$ for each (i, t) . Table 1 summarizes the action-level feasibility constraints enforced by the ILP framework. These constraints are described in detail in Appendix C. By enforcing feasibility before execution, ILP proactively prevents conflicts and invalid actions, rather than reacting to failures after they occur. The hyperparameter selection of

Table 1. Action-level constraints enforced by the ILP framework.

Constraint	Description	Formula / Key Info
Eligibility	Prevent physically impossible actions	$z_{i,t}^{(k)} = 0$ if $x_{i,t}^{(k)}$ is infeasible (invalid preconditions, blocked navigation.)
Per-Agent Action Count	Each agent executes exactly one action per planning step	$\sum_k z_{i,t}^{(k)} = 1 \quad \forall i$ (use no-op if necessary)
Multi-Agent Joint-Action	Ensure subtasks requiring multiple agents are fully staffed	$\sum_i \sum_{k \in X_{i,t}^{(s)}} z_{i,t}^{(k)} = k_s$
Resource Exclusivity	Prevent simultaneous use of exclusive resources	$\sum_i \sum_{k \in X_{i,t}^{(r)}} z_{i,t}^{(k)} \leq 1$
Collision / Interference	Avoid spatial or kinematic conflicts between agents	$z_{i,t}^{(k)} + z_{i',t}^{(k')} \leq 1$ for incompatible candidate pairs

action candidate count is discussed in Appendix D.5.4.

A Note on the ILP’s Scalability. MOSAIC mitigates ILP search-space explosion via *action pruning*, limiting the LLM-ACTOR to K candidates per agent; experiments (Appendix D.5.4) show the trade-offs related to the choice of K . For larger systems, *agent pruning* via hierarchical coordination or spatial partitioning can further cluster agents into local groups, keeping each ILP instance tractable.

Cost Function. Once feasibility constraints are satisfied, multiple joint actions may still be possible, differing in execution efficiency. Hence, the ILP framework uses a cost function to identify the suitable action per agent that minimizes the runtime cost of execution. The cost function considers spatial efficiency, temporal coherence, and workload balance by incorporating two weighted components:

$$\mathcal{J}(z, H) = \sum_{i=1}^N \sum_{k=1}^{K_i} z_{i,t}^{(k)} \left(C_{\text{penal}}(a_i, x_{i,t}^{(k)}, H) + \lambda_{\text{load}} C_{\text{load}}(a_i, x_{i,t}^{(k)}, H) \right). \quad (1)$$

The scaling coefficient λ_{load} balances the relative importance of load distribution versus penalties on failures, with a default value of 1.

Spatial and Temporal Penalties. C_{penal} captures common spatial and temporal inefficiencies, including cyclic behavior (repeated action sequences), repeated failures for a specific action, oscillatory back-and-forth spatial movements, backtracking to previously visited locations, and stagnation due to idle or no-progress steps. Using the agent’s action-success history H , these inefficiencies are incorporated as:

$$C_{\text{penal}}(a_i, x_{i,t}^{(k)}, H) = \sum_{p \in P} \lambda_p \cdot \text{Penalty}_p(a_i, x_{i,t}^{(k)}, H), \quad (2)$$

Each pattern $p \in P$ is scored using $\text{Penalty}_p(a_i, x_{i,t}^{(k)}, H_t)$ and weighted by λ_p . Detailed definitions of all penalty patterns and weights are provided in Appendix D.5.2 (Table 7).

Load Imbalance Penalty C_{load} promotes balanced workload by penalizing idle or underutilized agents:

$$C_{\text{load}}(a_i, x_{i,t}^{(k)}, H) = \frac{\frac{1}{|A|} \sum_{i' \in A} \text{ACTIONCOUNT}(a_{i'}, H)}{\text{ACTIONCOUNT}(a_i, H) + \mathbb{1}[x_{i,t}^{(k)} \neq \text{no-op}]},$$

where $\text{ACTIONCOUNT}(a_i, H)$ counts all non-idle actions executed by a_i up to step t . The numerator computes the average number of non-idle actions per agent, capturing overall team workload, while the denominator reflects agent a_i ’s cumulative workload, including the current action if non-idle. If a_i selects a `no-op`, the denominator remains unchanged, yielding a higher penalty relative to active agents. To handle the only possible zero-denominator case (an agent with no prior actions selecting `no-op`), we cap C_{load} at 3. This avoids unbounded penalties while preserving a strong incentive for agent utilization.

4. Experiments and Results

In this section, we first describe the experimental setup, followed by results demonstrating MOSAIC’s efficiency, effectiveness, and scalability.

4.1. Experimental Setup

Environments. We evaluate our framework in two environments: **AI2-THOR** (Apache 2.0) and **Search and Rescue (SAR)** (MIT) (Nayak et al., 2024). AI2-THOR is a photo-realistic 3D simulator for 36 multi-agent household tasks,

supporting up to 5 agents operating across kitchen, bedroom, living room, and bathroom. Agents perceive objects within a 1.5m range and a 90° field of view under partial observability, with optional noise in position, rotation, and detection sensors.

The SAR environment is a discrete, text-based grid simulator for emergency response missions. Agents navigate 2D grids to extinguish fires and rescue lost persons, managing limited visibility and dynamic conditions. Fires vary by intensity (low/medium/high) and type (chemical/non-chemical), requiring agents to collect appropriate resources (water or sand) from reservoirs. Rescue actions demand cooperative effort proportional to each person’s load, enforcing multi-agent coordination for resource allocation, fire control, and rescue under time constraints.

Consistent with the prior work, we assume reliable localization, as handling raw sensor drift lies outside our high-level planning scope and is typically addressed at the control layer. Nevertheless, we include a spatial noise sensitivity analysis in Appendix D.4 to assess robustness.

Models. MOSAIC uses one LLM to handle all LLM-PLANNER, LLM-ACTOR, and LLM-VERIFIER calls; this LLM is selected from **GPT-4o** (Hurst et al., 2024), **Claude Sonnet 4.5** (Anthropic, 2025), or **Gemini 3 Flash** (DeepMind / Google, 2025)¹. These hosted commercial models are used in accordance with their respective Terms of Service and exclusively for research and technical documentation purposes. For AI2-THOR, **CLIP ViT-B/32** (Radford et al., 2021), released under the MIT License and following the implementation used in **LLaMAR**, is additionally employed to compute visual-text similarity between agent camera observations and subtask descriptions, guiding agents toward task-relevant objects through optimal rotation or movement decisions.

Baselines. We categorize baselines by their per-planning-step complexity: **(a) Single-call per step baselines:** a single LLM invocation generates actions for all agents at each planning step. **(a1) ACT:** Direct action prediction, where a single LLM observes the environment and directly outputs actions for all agents at each planning step, without explicit reasoning or planning modules. **(a2) Chain-of-Thought (CoT)** (Wei et al., 2022): Prompts the LLM to reason step-by-step before outputting coordinated actions for all agents. **(a3) REACT** (Yao et al., 2022): A reasoning-and-acting approach in which the LLM explicitly outputs both a reasoning trace (Think) and agent actions (Action) at each step. **(b) Multi-call per step baselines:** employ multiple LLM invocations per planning step. **(b1) SMARTLLM** (Kannan et al., 2024): A hierarchical

¹All trademarks and logos are the property of their respective owners and are used here for identification and descriptive purposes only. No affiliation, sponsorship, or endorsement is implied.

planning approach that decomposes tasks into subtasks, assigns them to agents based on their skills and generates executable Python code. **(b2) CoELA** (Zhang et al., 2023b): A communication-enabled decentralized system where each agent independently generates messages and actions via separate LLM calls, coordinating through shared dialogue history. **(b3) LLAMAR** (Nayak et al., 2024): A Plan-Act-Verify architecture with explicit modules for planning, execution, failure correction, and progress verification. For the best-performing baselines, we evaluate variants that incorporate: (i) MOSAIC’s agent-centric semantic memory (§ 3.2), and (ii) MOSAIC’s ILP framework (§ 3.3).

Metrics. We evaluate MOSAIC along two dimensions: planning effectiveness (the first five metrics) and execution efficiency (the remaining four). Planning effectiveness metrics are bounded in $[0, 1]$. **(a) Success Rate:** fraction of episodes in which all tasks are completed. **(b) Transport Rate:** per-episode fraction of completed subtasks. **(c) Coverage:** fraction of successful interactions with task-relevant objects. **(d) Balance:** workload balance (B) across agents, $B = \min_i s_i / (\max_j s_j + \epsilon)$, where s_i is the number of successful actions by agent i , ϵ is 10^{-4} , and $i, j \in [n]$. $B = 1$ denotes perfect balance, $B \approx 0$ indicates at least one inactive agent. **(e) Failure Rate:** fraction of actions which could not be successfully executed, lower rate indicates better coordination among agents and conflict avoidance. **(f) Agent Steps:** total # of physical steps and primitive movements (e.g., pickup, putdown) across all agents per episode. A single symbolic action (e.g., moving between two distant locations) may consist of multiple agent steps. **(g) Runtime:** total elapsed wall-clock time per episode from start to termination, capturing overall execution latency. **(h) LLM Calls:** # of LLM invocations per episode (e.g., API calls or prompts), quantifying reasoning overhead independent of step duration. **(i) Prompt Token Count:** # of input tokens supplied to the LLM per episode, reflecting the communication overhead and memory load required for planning.

Results for all metrics are averaged over three episodes for all the tasks. Each episode is capped at 100 planning steps. Furthermore, agents execute actions synchronously at each step, and the system proceeds only after all agents complete their actions, ensuring coordinated updates.

4.2. Planning Effectiveness

Table 2 reports the planning effectiveness of MOSAIC and its baseline. For both AI2-THOR and SAR, MOSAIC consistently achieves the highest success and transport rates, maximum coverage and balanced workload, through maintaining the lowest failure rate.

MOSAIC achieves consistently the lowest failure rates. Baseline single-call planners such as ACT, CoT, and REACT rely on short-term textual memory limited to the cur-

rent object or destination. As a result, previously relevant but incomplete information is discarded, leading to redundant exploration and infeasible actions, with failure rates of 0.39–0.47 in AI2-THOR and 0.19–0.25 in SAR. Replacing short-term memory with ASM improves success rates by 1–3% points, but performance remains lower than multi-call LLAMAR, highlighting that task decomposition across multiple planning calls provides the largest boost in success. In the multi-call setting, SMARTLLM suffers from frequent errors in Python-based plan generation, while CoELA’s decentralized design causes context loss and inconsistent action selection, resulting in high failure rates of 0.46–0.51 in AI2-THOR and 0.23 in SAR. Stronger multi-call planners such as LLAMAR retain longer-term context as textual logs, but this memory remains unstructured due to absolute coordinates and orientations. This limits spatial reasoning during LLM inference, resulting in comparatively high failure rates (0.32 in AI2-THOR, 0.16 in SAR).

In contrast, MOSAIC’s ASM tracks all task-relevant objects and destinations, including those out of view, with the ILP framework enforcing feasibility constraints and penalizes conflicting or redundant actions, thereby reducing failures. When combined, ASM and ILP in MOSAIC reduce failures by 3% points for REACT and 7–12% points for LLAMAR. The improvements are particularly pronounced in the multi-call setting of LLAMAR, where task decomposition breaks complex goals into smaller subtasks, enabling more informed action selection and allocation.

MOSAIC achieves the highest task success rate because of reduced failures. Action execution failures can lead agents into patterns from which recovery is difficult. Such failures can prevent task completion and consequently reduce overall success rates. This effect is particularly pronounced for Python-based SMARTLLM, decentralized CoELA, and single-call method CoT, which exhibit higher failure rates (0.46–0.51 in AI2-THOR and 0.23–0.25 in SAR), corresponding to lower success rates (0.06–0.11 in AI2-THOR and 0.09–0.15 in SAR). CoT struggles on longer planning horizons, as it cannot maintain coherence across extended sequences, consistent with (Nayak et al., 2024; Stechly et al., 2024), which find it effective only with very specific prompts.

By reducing execution failures, MOSAIC enables agents to follow more feasible action trajectories, resulting in higher success rates across both environments under a fixed planning-step budget. In AI2-THOR, MOSAIC improves success from 0.44 to 0.51 (+7% points) in the single-call REACT setting and from 0.59 to 0.69 (+10% points) in the multi-call LLAMAR setting. For SAR, the improvements in success rate are from 0.55 to 0.63 (+8% points) for REACT and from 0.55 to 0.59 (+4% points) for LLAMAR. The larger gains for REACT in SAR are likely due to its shorter-

Table 2. Comparative evaluation of MOSAIC against baselines across effectiveness and efficiency metrics in **A12-THOR** and **SAR** environments. MOSAIC integrates *Agent-centric Semantic Memory (ASM)* and an *Integer Linear Programming (ILP)* framework. Arrows ($\uparrow/\downarrow$) indicate whether higher or lower values are better. The numbers in subscript indicate the change relative to REACT or LLAMAR. For the first five columns (effectiveness metrics), subscript is the absolute difference, while for the last four columns (efficiency metrics) subscript is percent difference. Positive improvements are highlighted in **green**, and drops are highlighted in **red**. Best results are highlighted in bold for both single-call and multi-call method classes. Standard deviation and variance are reported in Appendix D.7.

A12-THOR Environment		Effectiveness Metrics					Efficiency Metrics			
Method Class	Method	Success Rate ($\uparrow$)	Transport Rate ($\uparrow$)	Coverage ($\uparrow$)	Balance ($\uparrow$)	Failure Rate ($\downarrow$)	Runtime (Seconds, $\downarrow$)	# of LLM Calls ($\downarrow$)	# of Agent Steps ($\downarrow$)	# of Tokens ($\times 10^3$, $\downarrow$)
Single LLM call per planning step	ACT	0.42	0.63	0.82	0.70	0.42	398.0	156.2	251.1	1152
	CoT	0.08	0.29	0.43	0.62	0.47	229.1	62.4	346.2	501
	REACT	0.44	0.68	0.86	0.71	0.39	367.5	135.4	235.4	649
	$\hookrightarrow$ + ASM	0.47(+0.03)	0.72(+0.04)	0.90 (+0.04)	0.73(+0.02)	0.37(−0.02)	371.0(+0.9%)	130.6(− 3.5%)	230.3(− 2.2%)	722(+11.3%)
	$\hookrightarrow$ + ILP	0.49(+0.05)	0.71(+0.03)	0.90 (+0.04)	0.74(+0.03)	0.37(−0.02)	355.8(−3.2%)	124.8(− 7.8%)	211.3 (−10.3%)	729(+12.3%)
	$\hookrightarrow$ + MOSAIC	0.51 (+0.07)	0.73 (+0.05)	0.90 (+0.04)	0.76 (+0.05)	0.36 (−0.03)	342.0(−7.0%)	121.7(−10.1%)	216.1(− 8.3%)	724(+11.5%)
Multiple LLM calls per planning step	SMARTLLM	0.06	0.19	0.38	0.56	0.51	935.9	89.3	461.6	595
	CoELA	0.11	0.34	0.63	0.65	0.46	889.4	386.1	422.2	2953
	LLAMAR	0.59	0.85	0.90	0.79	0.32	757.9	346.8	295.8	1472
	$\hookrightarrow$ + ASM	0.61(+0.02)	0.87(+0.02)	0.91(+0.01)	0.82(+0.03)	0.29(−0.03)	723.8(− 4.5%)	310.2(−10.6%)	284.2(− 3.9%)	1562(+6.1%)
	$\hookrightarrow$ + ILP	0.67(+0.08)	0.90(+0.05)	0.93(+0.03)	0.85(+0.06)	0.24(−0.08)	628.1(−17.1%)	258.4(−25.5%)	240.3(−18.8%)	1420(−3.5%)
	$\hookrightarrow$ + MOSAIC	0.69 (+0.10)	0.92 (+0.07)	0.95 (+0.05)	0.87 (+0.08)	0.20 (−0.12)	554.1 (−26.9%)	231.5(−33.3%)	204.1 (−31.0%)	1364(−7.3%)
SAR Environment		Effectiveness Metrics					Efficiency Metrics			
Method Class	Method	Success Rate ($\uparrow$)	Transport Rate ($\uparrow$)	Coverage ($\uparrow$)	Balance ($\uparrow$)	Failure Rate ($\downarrow$)	Runtime (Seconds, $\downarrow$)	# of LLM Calls ($\downarrow$)	# of Agent Steps ($\downarrow$)	# of Tokens ($\times 10^3$, $\downarrow$)
Only one LLM call per planning step	ACT	0.08	0.15	0.43	0.79	0.25	51.9	84.4	163.3	437
	CoT	0.15	0.41	0.75	0.86	0.25	66.1	81.5	162.9	393
	REACT	0.55	0.76	0.89	0.87	0.19	47.4	76.3	95.3	321
	$\hookrightarrow$ + ASM	0.58(+0.03)	0.80(+0.04)	0.95(+0.06)	0.90(+0.03)	0.18(−0.01)	47.5(−0.04%)	74.0(− 2.9%)	89.6(− 6.0%)	371(+15.6%)
	$\hookrightarrow$ + ILP	0.62(+0.07)	0.81(+0.05)	0.95(+0.06)	0.93(+0.06)	0.18(−0.01)	44.9(−5.3%)	71.0(− 6.9%)	86.1(− 9.7%)	366(+14.0%)
	$\hookrightarrow$ + MOSAIC	0.63 (+0.08)	0.85 (+0.09)	0.95 (+0.06)	0.95 (+0.08)	0.16 (−0.03)	44.2 (−6.9%)	65.7 (−13.9%)	85.5 (−10.3%)	364(+13.4%)
Multiple LLM calls per planning step	SMARTLLM	0.12	0.34	0.51	0.82	0.23	513.3	101.6	146.5	486
	CoELA	0.09	0.32	0.49	0.82	0.23	545.3	383.7	191.9	2145
	LLAMAR	0.55	0.76	0.89	0.89	0.16	333.8	256.5	107.6	857
	$\hookrightarrow$ + ASM	0.55(−)	0.77(+0.01)	0.90(+0.01)	0.90(+0.01)	0.14(−0.02)	310.0(− 7.1%)	239.3(− 6.7%)	105.9(− 1.6%)	825(− 3.6%)
	$\hookrightarrow$ + ILP	0.58(+0.03)	0.78(+0.02)	0.91(+0.02)	0.92(+0.03)	0.10(−0.06)	250.4(−25.0%)	199.7(−22.2%)	88.9(−17.4%)	730(−14.7%)
	$\hookrightarrow$ + MOSAIC	0.59 (+0.04)	0.79 (+0.03)	0.92 (+0.03)	0.93 (+0.04)	0.09 (−0.07)	227.4 (−31.9%)	179.2(−30.2%)	80.6 (−25.1%)	683(−20.3%)

horizon and less complex environment, where structured memory and coordination allow immediate corrections and more effective action selection. Moreover, these improvements are not due to aggressive exploration or increased LLM usage, but rather by preventing execution failures.

4.3. Execution Efficiency

Table 2 additionally shows that MOSAIC reduces execution costs. Appendix D.2 further discusses effectiveness as a function of agent steps.

MOSAIC reduces runtime through lower failure rates. We observe that execution failures are the major contributor to high latency in embodied multi-agent planning. Each failed action triggers replanning, redundant steps, and extra LLM calls, compounding runtime. Consequently, methods with higher failure rates (ACT, CoT, SMARTLLM, CoELA) exhibit longer execution times within a fixed planning-step budget, increasing LLM usage without advancing task completion.

By reducing failure rates, MOSAIC directly limits inefficiencies such as redundant actions and additional LLM calls. In A12-THOR and SAR, MOSAIC reduces the failure rate

of REACT by 3% points, which corresponds to a runtime reduction of $\sim 7.0\%$, fewer LLM calls by 10.1–13.9%, and fewer agent steps 8.3–10.3%. In the multi-call LLAMAR setting, failure rates drop by 7–12% points, yielding a much larger runtime reduction 26.9–31.9%, alongside decreases in LLM calls 30.2–33.3% and agent steps 25.1–31.0%.

MOSAIC achieves the best tradeoff between efficiency and effectiveness. Single-call planners attain low runtime primarily by minimizing LLM calls, but this efficiency comes at the cost of poor action quality, and low task success. In A12-THOR, CoT achieves the lowest runtime (229.1s) and fewest LLM calls (62.4), yet requires 346.2 agent steps and succeeds in only 8% of tasks, whereas MOSAIC reduces agent steps to 204.1 (−41%) and increases success to 0.69 (+0.61), despite higher runtime (554.1s) and more LLM calls (231.5). A similar pattern appears in SAR, where ACT and CoT execute quickly (51.9s and 66.1s) but attain low success (0.08–0.15) and high failure rates (0.25), while MOSAIC on LLAMAR reaches 0.59 success with substantially fewer redundant actions.

Beyond outperforming single-call methods in effectiveness, MOSAIC also improves efficiency relative to strong multi-call baselines: compared to LLAMAR, across both envi-

Table 3. Performance and ILP overhead in the SAR environment across varying team sizes (3–8 agents). The ILP solver accounts for only 1–2% of total runtime and scales modestly, with negligible impact on overall latency.

Agent Count	Total Action-Agent Combinations	Method	Success Rate ($\uparrow$)	Failure Rate ($\downarrow$)	Total Runtime ($\downarrow$, s)	Total ILP Solver Runtime ($\downarrow$, s)	% of Total Runtime
3	9	LLAMAR	0.63	0.14	264.3	–	–
		$\hookrightarrow$ + MOSAIC	0.67	0.07	176.6	2.22	1.3
4	12	LLAMAR	0.65	0.12	269.9	–	–
		$\hookrightarrow$ + MOSAIC	0.68	0.05	188.5	2.66	1.4
5	15	LLAMAR	0.64	0.12	272.8	–	–
		$\hookrightarrow$ + MOSAIC	0.66	0.05	188.8	2.49	1.3
6	18	LLAMAR	0.62	0.16	280.3	–	–
		$\hookrightarrow$ + MOSAIC	0.65	0.08	199.1	3.34	1.7
7	21	LLAMAR	0.59	0.21	298.3	–	–
		$\hookrightarrow$ + MOSAIC	0.62	0.16	234.2	4.38	1.9
8	24	LLAMAR	0.51	0.26	319.1	–	–
		$\hookrightarrow$ + MOSAIC	0.55	0.19	301.2	5.87	1.9

ronments, it reduces runtime by 26.9–31.9%, LLM calls by 30.2–33.3%, and agent steps by 25.0–31.0%, and achieves consistent gains over REACT as well. While ASM increases token usage by 11.5–13.4% for REACT+MOSAIC, this modest overhead is outweighed by improvements in success rate (7–8% points), runtime reduction (7%), and decreased agent steps (8.3–10.3%).

4.4. Computation Scalability of ILP

Table 3 shows that ILP overhead remains consistently low, contributing only 1.3–1.9% of total runtime as agent count increases from 3 to 8. Although the number of action-agent combinations grows from 9 to 24, ILP solver runtime increases only modestly from 2.22s to 5.87s, indicating favorable scaling behavior. Across all team sizes, MOSAIC improves success rates by 2–4 points while reducing failure rates by up to 9 points relative to LLAMAR. Notably, despite introducing an additional optimization stage, MOSAIC also reduces overall runtime by 33–88 seconds in most settings. This efficiency is enabled by action pruning with a fixed $K=3$, with further scalability achievable through agent pruning in larger systems.

4.5. Additional Experiments

We provide detailed analyses of MOSAIC’s robustness, efficiency, coordination, and ablations in the appendices.

(a) Effects of agent counts. MOSAIC scales effectively with team size. In AI2-THOR with 3–5 agents, REACT+MOSAIC improves success by 15–20% points over REACT while reducing runtime by 15–25%, and LLAMAR+MOSAIC reaches up to 0.79 success rate with 20–30% faster execution compared to LLAMAR, yielding consistently better success–runtime tradeoffs. **(b) Effects of feasibility constraints and cost function.** Appendix D.5 highlights ILP constraints improve coordination (4–5% points

success increase) and the cost function boosts efficiency (12–22% fewer LLM calls, 12% faster runtime). **(c) LLM model choices.** Appendix D.3 shows that MOSAIC improves success across GPT-4o, Claude Sonnet 4.5, and Gemini 3 Flash (4–7% points for REACT, 10–11% points for LLAMAR) while reducing runtime by 27%, showing model-agnostic robustness. **(d) Effects of sensor noises.** In Appendix D.4, planning with MOSAIC remains effective under severe observation noise, with runtime at 88% of the noiseless setting. **(e) Effect of planning step budgets.** Appendix D.6 shows that increasing the planning budget from 50 to 100 steps raises success by 0.21–0.25, while further increases to 150–200 steps yield minimal gains (≤ 0.05) with higher runtime. **(f) Qualitative analysis.** Appendix E presents planning trajectories: LLAMAR exhibits cycles, repeated failures, and stagnation, whereas MOSAIC ensures fast, balanced, and consistently progressive execution.

5. Conclusion

MOSAIC demonstrates that runtime-efficient multi-agent planning can be achieved through lightweight agent-centric state tracking and constraint-guided coordination. Its core contributions, Agent-centric Semantic Memory (ASM) and Integer Linear Programming (ILP), address inefficiencies in state tracking and multi-agent execution. ASM enables compact spatial reasoning under partial observability, reducing failed actions by up to 12%, while ILP enforces step-level feasibility constraints over LLM proposals, achieving up to 32% faster execution, 33% fewer LLM calls, and 4–10% higher success rates. Across both AI2-THOR and search-and-rescue environments, these modules consistently reduce redundant actions and execution overhead, improving latency and task success. Overall, MOSAIC shows that structured memory and explicit feasibility constraints are effective plug-and-play mechanisms for scalable multi-agent embodied planning.

Acknowledgement

This material is based upon work supported by the National Science Foundation under Grant No. CNS-2312396, CNS-2338512, IIS-2435822, and CCF-2449995. Any opinions, findings, and conclusions or recommendations expressed in this material are those of the author(s) and do not necessarily reflect the views of the National Science Foundation.

Impact Statement

MOSAIC advances the practical deployment of LLM-based multi-agent systems in embodied environments by addressing the primary sources of inefficiency: failed actions, redundant exploration, and coordination breakdowns. By integrating lightweight, spatially grounded agent-centric memory with action-level joint optimization, the framework reduces execution latency and inference cost while maintaining high task success. Improved coordination efficiency limits unnecessary actions and resource waste, helping mitigate unsafe or unpredictable behavior and reducing environmental impact in shared environments. As more reliable multi-agent systems become viable for public-facing domains, these gains also highlight the growing need for transparent constraints and oversight as such systems increasingly shape real-world decisions.

Together, these advances enable operation under realistic time and resource constraints, narrowing the gap between research prototypes and deployable systems. The resulting improvements are especially impactful for safety- and time-critical applications such as search and rescue, household robotics, and long-horizon exploration, and demonstrate how principled integration of symbolic optimization with neural reasoning can unlock scalable, real-world multi-agent intelligence.

References

- Ahn, M., Brohan, A., Brown, N., Chebotar, Y., Cortes, O., David, B., Finn, C., Fu, C., Gopalakrishnan, K., Hausman, K., et al. Do as i can, not as i say: Grounding language in robotic affordances. *arXiv preprint arXiv:2204.01691*, 2022. 3, 15
- Anthropic. Claude sonnet 4.5 system card. System card, Anthropic, September 2025. URL <https://assets.anthropic.com/m/12f214efcc2f457a/original/Claude-Sonnet-4-5-System-Card.pdf>. 6
- Anwar, A., Welsh, J., Biswas, J., Pouya, S., and Chang, Y. Remembr: Building and reasoning over long-horizon spatio-temporal memory for robot navigation. In *2025 IEEE International Conference on Robotics and Automation (ICRA)*. IEEE, 2025. 3, 14
- Bai, D., Singh, I., Traum, D., and Thomason, J. Twostep: Multi-agent task planning using classical planners and large language models. *arXiv preprint 2403.17246*, 2025. 1
- Brawer, J., Bishop, K., Hayes, B., and Roncone, A. Towards a natural language interface for flexible multi-agent task assignment. In *Proceedings of the AAAI Symposium Series*, 2023. 2, 3, 15
- Cemri, M., Pan, M. Z., Yang, S., Agrawal, L. A., Chopra, B., Tiwari, R., Keutzer, K., Parameswaran, A., Klein, D., Ramchandran, K., Zaharia, M., Gonzalez, J. E., and Stoica, I. Why do multi-agent LLM systems fail? In *The Thirty-ninth Annual Conference on Neural Information Processing Systems Datasets and Benchmarks Track*, 2026. 3, 14
- Chaplot, D. S., Gandhi, D., Gupta, S., Gupta, A., and Salakhutdinov, R. Learning to explore using active neural slam. *arXiv preprint arXiv:2004.05155*, 2020. 3, 14
- Chen, J., Hu, X., Liu, S., Huang, S., Tu, W.-W., He, Z., and Wen, L. LLMarena: Assessing capabilities of large language models in dynamic multi-agent environments. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. Association for Computational Linguistics, 2024. 1
- Choe, D. B., Sangeetha, S. V., Emanuel, S., Chiu, C.-Y., Coogan, S., and Kousik, S. Ask, reason, assist: Decentralized robot collaboration via language and logic. *arXiv preprint arXiv:2509.23506*, 2025. 2, 3, 15
- Cornelio, C. and Diab, M. Recover: A neuro-symbolic framework for failure detection and recovery. In *2024 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*. IEEE, 2024. 3, 15
- Dantam, N. T., Kingston, Z. K., Chaudhuri, S., and Kavraki, L. E. An incremental constraint-based framework for task and motion planning. *The International Journal of Robotics Research*, 2018. 3, 15
- DeepMind / Google. Gemini 3 flash model card. Model card / technical report, Google DeepMind, December 2025. URL <https://storage.googleapis.com/deepmind-media/Model-Cards/Gemini-3-Flash-Model-Card.pdf>. 6
- Fang, K., Toshev, A., Fei-Fei, L., and Savarese, S. Scene memory transformer for embodied agents in long-horizon tasks. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, pp. 538–547, 2019. 2, 3, 14

- Faroni, M., Umbrico, A., Beschi, M., Orlandini, A., Cesta, A., and Pedrocchi, N. Optimal task and motion planning and execution for multiagent systems in dynamic environments. *IEEE Transactions on Cybernetics*, 2024. 3, 15
- Garrett, C. R., Chitnis, R., Holladay, R., Kim, B., Silver, T., Kaelbling, L. P., and Lozano-Pérez, T. Integrated task and motion planning. *Annual review of control, robotics, and autonomous systems*, 2021. 3, 15
- Gu, Q., Kuwajerwala, A., Morin, S., Jatavallabhula, K. M., Sen, B., Agarwal, A., Rivera, C., Paul, W., Ellis, K., Chellappa, R., et al. Conceptgraphs: Open-vocabulary 3d scene graphs for perception and planning. In *2024 IEEE International Conference on Robotics and Automation (ICRA)*. IEEE, 2024. 3, 14
- Gupta, G., Yadav, K., Kira, Z., Gal, Y., and Aljundi, R. Memo: Training memory-efficient embodied agents with reinforcement learning. *arXiv preprint arXiv:2510.19732*, 2025a. 3, 14
- Gupta, N., Srinivasaraghavan, G., Mohalik, S., Kumar, N., and Taylor, M. E. Hammer: Multi-level coordination of reinforcement learning agents via learned messaging. *Neural Computing and Applications*, 2025b. 14
- Gupta, S., Davidson, J., Levine, S., Sukthankar, R., and Malik, J. Cognitive mapping and planning for visual navigation. In *Proceedings of the IEEE conference on computer vision and pattern recognition*, 2017. 3, 14
- Han, S., Dastani, M., and Wang, S. Sparse communication in multi-agent deep reinforcement learning. *Neurocomputing*, 625:129344, 2025a. 14
- Han, S., Zhang, Q., Yao, Y., Jin, W., and Xu, Z. Llm multi-agent systems: Challenges and open problems. *arXiv preprint 2402.03578*, 2025b. 3, 14
- He, L., Dong, H., Chen, Z., Yu, Y., Feng, J., and Li, Y. Mem4nav: Boosting vision-and-language navigation in urban environments with a hierarchical spatial-cognition long-short memory system. *arXiv preprint arXiv:2506.19433*, 2025. 2, 3, 14
- Hill, B. A., Wei, M. K. E., and Jishnuanandh, T. Communicating plans, not percepts: Scalable multi-agent coordination with embodied world models. *arXiv preprint arXiv:2508.02912*, 2025. 14
- Hu, M., Chen, T., Chen, Q., Mu, Y., Shao, W., and Luo, P. Hiagent: Hierarchical working memory management for solving long-horizon agent tasks with large language model. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, 2025a. 3, 14
- Hu, W., Hong, Y., Wang, Y., Gao, L., Wei, Z., Yao, X., Peng, N., Bitton, Y., Szpektor, I., and Chang, K.-W. 3D LLM-mem: Long-term spatial-temporal memory for embodied 3d large language model. In *The Thirty-ninth Annual Conference on Neural Information Processing Systems*, 2025b. 2, 3, 14
- Huang, C., Mees, O., Zeng, A., and Burgard, W. Visual language maps for robot navigation. In *Proceedings of the IEEE International Conference on Robotics and Automation (ICRA)*, 2023. 3, 14
- Huang, W., Xia, F., Xiao, T., Chan, H., Liang, J., Florence, P., Zeng, A., Tompson, J., Mordatch, I., Chebotar, Y., et al. Inner monologue: Embodied reasoning through planning with language models. *arXiv preprint arXiv:2207.05608*, 2022. 3, 15
- Hurst, A., Lerer, A., Goucher, A. P., Perelman, A., Ramesh, A., Clark, A., Ostrow, A., Welihinda, A., Hayes, A., Radford, A., et al. Gpt-4o system card. *arXiv preprint 2410.21276*, 2024. 6
- Ji, Z., Wu, D., Ma, P., Li, Z., and Wang, S. Testing and understanding erroneous planning in llm agents through synthesized user inputs. *arXiv preprint arXiv:2404.17833*, 2024. 3, 15
- Kaelbling, L. P. and Lozano-Pérez, T. Hierarchical task and motion planning in the now. In *2011 IEEE international conference on robotics and automation*, pp. 1470–1477. IEEE, 2011. 3, 15
- Kannan, S. S., Venkatesh, V. L., and Min, B.-C. Smart-llm: Smart multi-agent robot task planning using large language models. In *2024 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*. IEEE, 2024. 3, 4, 6, 15
- Kolve, E., Mottaghi, R., Han, W., VanderBilt, E., Weihs, L., Herrasti, A., Gordon, D., Zhu, Y., Gupta, A., and Farhadi, A. AI2-THOR: An Interactive 3D Environment for Visual AI. *arXiv preprint 1712.05474*, 2017. 2
- Lei, M., Cai, H., Cui, Z., Tan, L., Hong, J., Hu, G., Zhu, S., Wu, Y., Jiang, S., Wang, G., et al. Robomemory: A brain-inspired multi-memory agentic framework for lifelong learning in physical embodied systems. In *NeurIPS 2025 Workshop on Space in Vision, Language, and Embodied AI*, 2025a. 3, 14
- Lei, M., Zhao, Y., Wang, G., Mai, Z., Cui, S., Han, Y., and Ren, J. Stma: A spatio-temporal memory agent for long-horizon embodied task planning. *arXiv preprint arXiv:2502.10177*, 2025b. 3, 14
- Li, X., Wang, S., Zeng, S., Wu, Y., and Yang, Y. A survey on llm-based multi-agent systems: workflow, infrastructure, and challenges. *Vicinagearth*, 2024. 3, 14

- Lin, F., Chen, S., Fang, R., Wang, H., and Lin, T. Stop wasting your tokens: Towards efficient runtime multi-agent systems. *arXiv preprint 2510.26585*, 2025. 2, 3, 14
- Liu, X., Guo, D., Zhang, X., and Liu, H. Heterogeneous embodied multi-agent collaboration. *IEEE Robotics and Automation Letters*, 9(6):5377–5384, 2024. doi: 10.1109/LRA.2024.3390588. 1
- Lozano-Pérez, T. and Kaelbling, L. P. A constraint-based method for solving sequential manipulation planning problems. In *2014 IEEE/RSJ International Conference on Intelligent Robots and Systems*. IEEE, 2014. 3, 15
- Mandi, Z., Jain, S., and Song, S. Roco: Dialectic multi-robot collaboration with large language models. In *2024 IEEE International Conference on Robotics and Automation (ICRA)*. IEEE, 2024. 14
- Mao, Y., Ye, H., Dong, W., Zhang, C., and Zhang, H. Meta-memory: Retrieving and integrating semantic-spatial memories for robot spatial reasoning. *arXiv preprint arXiv:2509.20754*, 2025. 2, 3, 14
- Nayak, S., Orozco, A. M., Have, M. T., Zhang, J., Thirumalai, V., Chen, D., Kapoor, A., Robinson, E., Gopalakrishnan, K., Harrison, J., Mahajan, A., brian ichter, and Balakrishnan, H. Long-horizon planning for multi-agent robots in partially observable environments. In *The Thirty-eighth Annual Conference on Neural Information Processing Systems*, 2024. 1, 2, 3, 4, 6, 7, 14, 15, 35
- Nomura, K., Aoki, T., Taniguchi, T., and Horii, T. Decentralized collective world model for emergent communication and coordination. *arXiv preprint arXiv:2504.03353*, 2025. 14
- Obata, K., Aoki, T., Horii, T., Taniguchi, T., and Nagai, T. Lip-llm: Integrating linear programming and dependency graph with large language models for multi-robot task planning. *IEEE Robotics and Automation Letters*, 2024. 2, 3, 15
- Owerko, D., Vatnsdal, F., Agarwal, S., Kumar, V., and Ribeiro, A. Mast: Multi-agent spatial transformer for learning to collaborate. *arXiv preprint arXiv:2509.17195*, 2025. 14
- Pesce, E. and Montana, G. Improving coordination in small-scale multi-agent deep reinforcement learning through memory-driven communication. *Machine Learning*, 2020. 14
- Qian, C., Xie, Z., Wang, Y., Liu, W., Zhu, K., Xia, H., Dang, Y., Du, Z., Chen, W., Yang, C., Liu, Z., and Sun, M. Scaling large language model-based multi-agent collaboration. In *International Conference on Representation Learning*, 2025. 1
- Radford, A., Kim, J. W., Hallacy, C., Ramesh, A., Goh, G., Agarwal, S., Sastry, G., Askell, A., Mishkin, P., Clark, J., et al. Learning transferable visual models from natural language supervision. In *International conference on machine learning*. PmLR, 2021. 6
- Simões, D., Lau, N., and Reis, L. P. Multi-agent actor centralized-critic with communication. *Neurocomputing*, 2020. 14
- Skrynnik, A., Andreychuk, A., Yakovlev, K., and Panov, A. Decentralized monte carlo tree search for partially observable multi-agent pathfinding. In *Proceedings of the AAAI Conference on Artificial Intelligence*, 2024. 1
- Stechly, K., Valmeekam, K., and Kambhampati, S. Chain of thoughtlessness? an analysis of cot in planning. In *The Thirty-eighth Annual Conference on Neural Information Processing Systems*, 2024. 7
- Tan, H., Hao, X., Chi, C., Lin, M., Lyu, Y., Cao, M., Liang, D., Chen, Z., Lyu, M., Peng, C., et al. Ro-boos: A hierarchical embodied framework for cross-embodiment and multi-agent collaboration. *arXiv preprint arXiv:2505.03673*, 2025. 14
- Wang, G., Xie, Y., Jiang, Y., Mandlkar, A., Xiao, C., Zhu, Y., Fan, L., and Anandkumar, A. Voyager: An open-ended embodied agent with large language models. *arXiv preprint arXiv:2305.16291*, 2023. 3, 15
- Wang, Z., Yu, B., Zhao, J., Sun, W., Hou, S., Liang, S., Hu, X., Han, Y., and Gan, Y. Karma: Augmenting embodied ai agents with long-and-short term memory systems. In *2025 IEEE International Conference on Robotics and Automation (ICRA)*. IEEE, 2025. 3, 14
- Wei, J., Wang, X., Schuurmans, D., Bosma, M., Ichter, B., Xia, F., Chi, E. H., Le, Q. V., and Zhou, D. Chain-of-thought prompting elicits reasoning in large language models. In *Proceedings of the 36th International Conference on Neural Information Processing Systems*, NIPS ’22, 2022. 6
- Wu, P., Mu, Y., Zhou, K., Ma, J., Chen, J., and Liu, C. Camon: Cooperative agents for multi-object navigation with llm-based conversations. *arXiv preprint arXiv:2407.00632*, 2024a. 14
- Wu, Y., Tao, Y., Li, P., Shi, G., Sukhatme, G. S., Kumar, V., and Zhou, L. Hierarchical llms in-the-loop optimization for real-time multi-robot target tracking under unknown hazards. *arXiv preprint arXiv:2409.12274*, 2024b. 2, 3, 15
- Yadla, P. Temporal multimodal memory banks for agentic reasoning. In *Proceedings of the IEEE/CVF International Conference on Computer Vision*, 2025. 3, 14

- Yang, Y., Yang, H., Zhou, J., Chen, P., Zhang, H., Du, Y., and Gan, C. 3d-mem: 3d scene memory for embodied exploration and reasoning. In *Proceedings of the Computer Vision and Pattern Recognition Conference*, 2025. 3, 14
- Yao, S., Zhao, J., Yu, D., Du, N., Shafran, I., Narasimhan, K. R., and Cao, Y. React: Synergizing reasoning and acting in language models. In *The eleventh international conference on learning representations*, 2022. 2, 3, 6, 14, 15
- Ye, Y. Task memory engine: Spatial memory for robust multi-step llm agents. *arXiv preprint arXiv:2505.19436*, 2025. 3, 14
- Ying, L., Jha, K., Aarya, S., Tenenbaum, J. B., Torralba, A., and Shu, T. Goma: Proactive embodied cooperative communication via goal-oriented mental alignment. In *2024 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2024. 14
- Zhang, G., Fu, M., Wan, G., Yu, M., Wang, K., and Yan, S. G-memory: Tracing hierarchical memory for multi-agent systems. *arXiv preprint 2506.07398*, 2025a. 3, 14
- Zhang, H., Du, W., Shan, J., Zhou, Q., Du, Y., Tenenbaum, J. B., Shu, T., and Gan, C. Building cooperative embodied agents modularly with large language models. *arXiv preprint arXiv:2307.02485*, 2023a. 14
- Zhang, H., Du, W., Shan, J., Zhou, Q., Du, Y., Tenenbaum, J. B., Shu, T., and Gan, C. Building cooperative embodied agents modularly with large language models. *arXiv preprint 2307.02485*, 2023b. 1, 3, 4, 7, 15
- Zhang, H., Zhang, Z., Wang, Z., Zhang, Z., Fang, L., Zhou, Q., and Gan, C. Ella: Embodied social agents with life-long memory. *arXiv preprint arXiv:2506.24019*, 2025b. 3, 14
- Zhang, L., Liu, Y., Zhang, Z., Aghaei, M., Hu, Y., Gu, H., Alomrani, M. A., Bravo, D. G. A., Karimi, R., Hamidizadeh, A., Xu, H., Huang, G., zhanpeng zhang, Cao, T., Qiu, W., Quan, X., HAO, J., Zhuang, Y., and Zhang, Y. Mem2ego: Empowering vision-language models with global-to-ego memory for long-horizon embodied navigation. In *Workshop on Foundation Models Meet Embodied Agents at CVPR 2025*, 2025c. 3, 14
- Zhang, S., Yin, M., Zhang, J., Liu, J., Han, Z., Zhang, J., Li, B., Wang, C., Wang, H., Chen, Y., and Wu, Q. Which agent causes task failures and when? on automated failure attribution of llm multi-agent systems. *arXiv preprint 2505.00212*, 2025d. 3, 14
- Zhang, X., Qin, H., Wang, F., Dong, Y., and Li, J. Lamma-p: Generalizable multi-agent long-horizon task allocation and planning with lm-driven pddl planner. In *2025 IEEE International Conference on Robotics and Automation (ICRA)*. IEEE, 2025e. 1
- Zheng, Z., Feng, Q., Li, H., Knoll, A., and Feng, J. Evaluating uncertainty-based failure detection for closed-loop llm planners. *arXiv preprint arXiv:2406.00430*, 2024. 3, 15

A. Extended Related Work

We review prior work on memory representations for embodied planning, as well as approaches for coordinating actions among multiple agents.

Memory Representations for Embodied Agents. Work on memory for embodied agents can be broadly categorized into (a) **temporal**, (b) **spatial**, and (c) **spatio-temporal memory**.

(a) **Temporal memory** retains past observations over time without explicitly storing object locations or geometry. It includes (i) *short-term memory*, which stores the most recent observations or latent states over a few timesteps via limited context windows (Yao et al., 2022; Lin et al., 2025); (ii) *long-term or episodic memory*, which preserves information across extended horizons such as full task episodes or multiple trajectories using persistent memory modules or databases (Nayak et al., 2024; Fang et al., 2019; Gupta et al., 2025a; Lei et al., 2025a; Zhang et al., 2025b; Yadla, 2025); and (iii) *hierarchical memory*, which organizes temporal information at multiple abstraction levels (e.g., low-level observations, subgoals, high-level plans) (Wang et al., 2025; Hu et al., 2025a; Lei et al., 2025b; Zhang et al., 2025a; Han et al., 2025b). While effective for retaining history, temporal-only memories often rely on unstructured textual logs (Cemri et al., 2026; Zhang et al., 2025d; Li et al., 2024), requiring the LLM to infer object relationships and positions from text alone, which is error-prone for spatial reasoning.

(b) **Spatial memory** explicitly stores the layout of the environment (such as object locations, or free space) independent of time, enabling direct spatial reasoning; representative approaches include classical topology-based or sensor-based navigation systems (Gupta et al., 2017; Chaplot et al., 2020) and recent dense 3D representations such as VLMAPS (Huang et al., 2023), CONCEPTGRAPHS (Gu et al., 2024), MEM4NAV (He et al., 2025), TME (Ye, 2025), and MEM2EGO (Zhang et al., 2025c). In contrast to these dense spatial representations, which often incur significant overhead to store high-fidelity geometry, our memory focuses on an agent-centric abstraction of task-relevant objects and destinations, prioritizing runtime-efficient tracking over complete environmental reconstruction.

(c) **Spatio-temporal memory** combines both dimensions by maintaining spatial representations that evolve over time, allowing LLMs to reason about dynamic environments in which objects move, appear, or disappear, as explored in SCENE MEMORY TRANSFORMER (Fang et al., 2019), 3D-MEM (Yang et al., 2025), STMA (Lei et al., 2025b), META-MEMORY (Mao et al., 2025), REMEMBR (Anwar et al., 2025), and 3DLLM-MEM (Hu et al., 2025b). Constructing and updating spatio-temporal memories typically requires accumulating observations over long horizons and synchronizing rich representations, which introduces significant computational and latency overhead. In contrast, our approach adopts an agent-centric spatio-temporal memory in which each agent maintains a structured textual representation of task-relevant objects and destinations relative to itself. This memory is updated at every planning step to track relative distances and goals (even when objects fall outside the agent’s current field of view) enabling efficient spatial grounding in dynamic and partially-observable environments.

Multi-Agent Coordination. Multi-agent coordination methods can be characterized along two key dimensions: (i) **Coordination Topology**, distinguishing centralized approaches from decentralized, message-passing ones, and (ii) **Coordination Mechanism**, which determines how task and action allocation decisions are produced, e.g., via symbolic planners or neural policies. **Failure handling** is closely tied to the coordination mechanism: symbolic methods can proactively anticipate conflicts and infeasible plans, while neural approaches are largely reactive, correcting errors only after execution.

(a) **Coordination Topology: Centralized vs. Decentralized Message Passing.** Coordination can be *decentralized*, where agents exchange information through peer-to-peer communication under bandwidth or observability constraints (Zhang et al., 2023a; Ying et al., 2024; Wu et al., 2024a; Nomura et al., 2025; Owerko et al., 2025; Pesce & Montana, 2020; Gupta et al., 2025b; Han et al., 2025a), or *centralized*, where a shared entity aggregates information to maintain consistent knowledge of the environment and agent actions (Simões et al., 2020; Tan et al., 2025; Mandi et al., 2024; Hill et al., 2025). We adopt centralized message passing, as we focus on collaborative settings (such as search-and-rescue and household tasks) where inter-agent privacy concerns are limited and maximizing coordination is critical for task success.

(b) **Coordination Mechanism: Task/Action Allocation and Failure Management.** Coordination mechanisms differ in how task and action allocation decisions are produced, ranging from symbolic planners, which rely on explicit constraints, to neural policies, which learn to suggest actions from data. Alongside allocation, these mechanisms differ in how they handle execution failures: some methods are reactive, correcting conflicts or infeasible actions only after they occur, while others are proactive, anticipating and preventing failures before execution.

Neural methods typically propose candidate actions or task assignments based on learned experience, often requiring additional mechanisms to ensure feasibility and resolve failures. They are largely reactive, correcting errors only after the actions are executed (Yao et al., 2022; Kannan et al., 2024; Nayak et al., 2024; Zhang et al., 2023b; Wang et al., 2023; Ahn et al., 2022; Ji et al., 2024; Huang et al., 2022). Symbolic approaches leverage preconditions and logical rules to proactively reason about feasibility and reduce execution failures (Zheng et al., 2024; Cornelio & Diab, 2024), and are commonly instantiated through Task and Motion Planning (TAMP) frameworks or LLM-assisted optimization layers (Faroni et al., 2024; Kaelbling & Lozano-Pérez, 2011; Garrett et al., 2021; Lozano-Pérez & Kaelbling, 2014; Dantam et al., 2018). Existing neuro-symbolic coordination mechanisms have limited scope. Choe et al., 2025 handles failures reactively, allowing warehouse agents to request or offer help via symbolic constraints. Brawer et al., 2023 checks feasibility among user-specified actions but does not aim for runtime optimization. LiP-LLM (Obata et al., 2024) and Wu et al. 2024b assign tasks to agents while ensuring no conflicts over target objects using symbolic constraints. However, these methods do not perform action-level assignments, which are necessary to reason about conflicts beyond target object allocation at each planning step.

In contrast, MOSAIC uses a neuro-symbolic, **runtime optimization-based coordination mechanism** that performs **joint action-level assignments** at every timestep, guided by LLM-generated action suggestions. This enables **proactive failure avoidance** through fine-grained action feasibility reasoning.

B. Multi-Agent Planning Architecture

Algorithm 1 General Architecture Multi-Agent Embodied Planning

Require: Task instruction I , Action constraints C , Agents $A = \{a_1, \dots, a_N\}$, Architecture mode $\text{Mode} \in \{\text{Centralized}, \text{Decentralized}\}$
Ensure: Executed multi-agent plan

- 1: **if** $\text{Mode} = \text{Centralized}$ **then**
- 2: **initialize** global memory $M \leftarrow \emptyset$
- 3: **else**
- 4: **for** each agent $a_i \in A$ **do**
- 5: **initialize** local memory $M_i \leftarrow \emptyset$
- 6: **end for**
- 7: **end if**
- 8: **initialize** $H \leftarrow \emptyset$ ▷ Per-agent action-success history
- 9: **initialize** $S_o \leftarrow \emptyset$ ▷ Open subtasks
- 10: **initialize** $S_c \leftarrow \emptyset$ ▷ Completed subtasks
- 11: $S_o \leftarrow \text{LLM-PLANNER}(I, S_o, S_c, M \text{ if Centralized else } \{M_i\}_{i=1}^N)$ ▷ Initial decomposition into subtasks using appropriate memory
- 12: **while** $S_o \neq \emptyset$ **do**
- 13: **if** $\text{Mode} = \text{Centralized}$ **then**
- 14: $\{x_i\}_{i=1}^N \leftarrow \text{LLM-ACTOR}(A, C, S_o, H, M)$ ▷ Single LLM call produces actions for all agents
- 15: **else**
- 16: **for** each agent $a_i \in A$ **do**
- 17: $x_i \leftarrow \text{LLM-ACTOR}(a_i, C, S_o, H, M_i)$ ▷ Each agent calls its own LLM
- 18: **end for**
- 19: **end if**
- 20: $H, M \leftarrow \text{EXECUTEINPARALLEL}(\{(a_i, x_i)\}_{i=1}^N)$ ▷ Execute actions and update history/memory
- 21: $S_o, S_c \leftarrow \text{LLM-VERIFIER}(S_o, S_c, H, M \text{ if Centralized else } \{M_i\}_{i=1}^N)$ ▷ Check which subtasks are completed
- 22: $S_o \leftarrow \text{LLM-PLANNER}(I, S_o, S_c, M \text{ if Centralized else } \{M_i\}_{i=1}^N)$ ▷ Optional re-planning based on updated memory
- 23: **end while**

Algorithm 1 describes a general planning loop for both centralized and decentralized multi-agent execution, in which different LLMs are used to (a) decompose high-level instructions into subtasks, (b) propose actions for each of the subtasks, and (c) verify subtask completion. This separation of roles follows prior LLM-based embodied planning systems (Nayak et al., 2024; Zhang et al., 2023b; Kannan et al., 2024), which decouple *high-level reasoning* (task decomposition and progress assessment) from *low-level decision making* (action selection) to improve execution reliability and task success.

The algorithm begins by initializing memory according to the selected mode (lines 1–7). In centralized mode, a single global memory M is maintained and shared across all agents, whereas in decentralized mode, each agent maintains its own local memory M_i . This ensures that LLMs can reason using either a shared or independent knowledge base depending on the architecture. Simultaneously, the interaction history H and bookkeeping variables for open and completed subtasks, S_o and S_c , are initialized (lines 8–10) to record prior actions, execution outcomes, and task progress, providing context for planning and verification.

Given the high-level instruction I , the LLM-PLANNER produces an initial decomposition into subtasks, conditioned on the available memory (line 11), allowing the system to convert natural language goals into trackable objectives. The algorithm then proceeds iteratively until all subtasks are completed (line 12), with each iteration corresponding to one execution step. In each step, agent actions are generated via LLM-ACTOR (lines 13–19), which is responsible for translating the current task context into concrete, executable actions. In centralized mode, a single LLM call produces actions for all agents jointly, enabling globally consistent coordination. In decentralized mode, each agent calls its own LLM to propose an action based on its local memory. In both cases, action generation considers the current set of open subtasks, the accumulated execution history, and action constraints, allowing the LLM to reason at the action level for fine-grained coordination.

Once proposed, the actions are executed in parallel, and the resulting outcomes are used to update both the interaction history and the relevant memory representations (line 20). The LLM-VERIFIER then evaluates which subtasks have been successfully completed and updates S_o and S_c accordingly (line 21). This verification is critical because completing a subtask may change which subsequent actions or subtasks are relevant, depending on how the environment responded to the agents’ actions. The LLM-PLANNER may then be re-invoked to revise the remaining subtasks in light of this updated context (line 22), ensuring that planning remains adaptive to partial observability, execution uncertainty, and dynamic environments while maintaining a consistent framework across both centralized and decentralized architectures.

C. Action-level Feasibility Constraints

We detail the ILP’s action-level feasibility constraints that regulate candidate actions to prevent conflicts, infeasible assignments, and execution failures.

Eligibility Constraints. If an action candidate $x_{i,t}^{(k)}$ is physically infeasible for agent a_i at step t (e.g., attempting to place or manipulate an object that is not currently held, navigating into an occupied or non-traversable region such as a wall, or violating basic action preconditions), we enforce $z_{i,t}^{(k)} = 0$. This prevents the solver from considering impossible action assignments.

Per-Agent Action Selection. At each planning step t , each agent is required to execute exactly one action, enforced by $\sum_k z_{i,t}^{(k)} = 1 \quad \forall i$. If no meaningful action is available, the candidate set $X_{i,t}$ includes a special `no-op` or `done` action, ensuring the constraint remains satisfiable. This guarantees that the joint action at each step is well-defined and avoids repeated re-planning or inconsistent partial assignments.

Multi-Agent Joint-Action Requirements. Some subtasks require multiple agents to act simultaneously (e.g., carrying a heavy object or assisting a person). Let s denote such a subtask that requires k_s agents, and let $X_{i,t}^{(s)} \subseteq X_{i,t}$ denote the subset of action candidates for agent a_i at step t that correspond to subtask s . We enforce $\sum_i \sum_{k \in X_{i,t}^{(s)}} z_{i,t}^{(k)} = k_s$. This ensures that joint subtasks are neither under-staffed, which would lead to execution failure, nor over-staffed, which would waste actions and increase runtime.

Resource Exclusivity. When multiple action candidates require exclusive access to the same resource (e.g., an object, tool, or environment element), we impose $\sum_i \sum_{k \in X_{i,t}^{(r)}} z_{i,t}^{(k)} \leq 1$, where $X_{i,t}^{(r)} \subseteq X_{i,t}$ denotes the set of action candidates for agent a_i at step t that require resource r . This prevents conflicts such as simultaneous grasps of the same object, concurrent attempts to open the same door, or contention over shared tools.

Collision and Interference Avoidance. Certain action candidates may be mutually incompatible due to spatial or kinematic constraints. For any pair of incompatible action candidates $(x_{i,t}^{(k)}, x_{i',t}^{(k')})$, we enforce $z_{i,t}^{(k)} + z_{i',t}^{(k')} \leq 1$. This prevents collisions, overlapping target locations, and other mutually exclusive spatial configurations across agents at the same step.

D. Additional Results

This section presents additional experiments that further analyze the behavior, robustness, and generality of our approach. We examine how performance scales with the number of agents, characterize effectiveness trends across planning steps, and evaluate generalization across different models. We additionally study robustness under observation noise, isolate the contributions of individual components of the ILP framework through ablations, analyze the impact of the planning step budget, and report performance variance to assess stability.

D.1. Scalability of MOSAIC

We evaluate scalability by varying agent count (1–5) and measuring success rate and runtime in AI2-THOR (Figure 3a) and SAR (Figure 3b). The corresponding numerical results, along with failure rates, are listed in Table 4.

MOSAIC improves efficiency and effectiveness as agent count increases to 3. Across both environments, augmenting existing planners with MOSAIC increases success rates while reducing execution time, with gains preserved as the number of agents grows. In AI2-THOR, REACT+MOSAIC improves success over REACT by 15–20% points at higher agent counts (3–5), while LLAMAR+MOSAIC gains 10–14% points and achieves the highest overall success (up to 0.79). In SAR, MOSAIC demonstrates similar benefits: integrating MOSAIC with REACT raises success from 28% to 38% for a single agent (+10% points) and from 65% to 68% for four agents (+3% points). For LLAMAR, success improves from 29% to 31% (+2% points) for one agent and from 65% to 68% (+3% points) for four agents.

These improvements are accompanied by consistent runtime reductions. In AI2-THOR, REACT+MOSAIC reduces runtime by 15–25% across agent counts, while LLAMAR+MOSAIC achieves 20–30% reductions. In SAR, REACT+MOSAIC completes tasks in 35–44s vs 37–49s for the baseline (10–30% faster), and LLAMAR+MOSAIC executes in 177–189s compared to 264–343s for LLAMAR alone (30–32% faster). Gains are most pronounced for intermediate team sizes (2–3 agents), indicating that MOSAIC effectively reduces coordination bottlenecks as team size increases.

Effectiveness saturates gracefully with 4–5 agents. Increasing agent count yields large initial gains in success rate, especially for MOSAIC-based methods. From 1 to 3 agents, REACT+MOSAIC improves success by 25% points in AI2-THOR (0.38→0.63) and 29% points in SAR (0.38→0.67), while LLAMAR+MOSAIC gains 34% points in AI2-THOR (0.45→0.79) and 36% points in SAR (0.31→0.67). Beyond this range, performance saturates: from 3 to 5 agents, success varies by at most 2–3 points in AI2-THOR and 1–2 points in SAR. Notably, MOSAIC saturates at higher asymptotic success (0.76–0.79 in AI2-THOR and 0.66–0.68 in SAR) than non-MOSAIC baselines (0.63–0.68 and 0.63–0.65). This saturation likely reflects the spatial scale of the evaluated environments; in larger or more complex settings, additional agents are expected to yield more sustained, near-linear gains.

Overall, MOSAIC ensures favorable scaling behavior: higher success rates, reduced execution time, and graceful performance saturation as agent count increases, demonstrating that coordinated planning with ASM and ILP supports both effective and runtime-efficient multi-agent operation.

D.2. Effectiveness Trends

Figures 4 and 5 show the evolution of transport rate, balance, and coverage over planning steps for household rearrangement and search-and-rescue tasks.

Balance correlates with transport efficiency. Across methods, improvements in balance closely track gains in transport rate. Planners that rapidly achieve high balance (such as REACT + MOSAIC and LLAMAR + MOSAIC) also exhibit the fastest rise in transport rate. This indicates that equitable workload distribution is not merely a fairness metric, but a key enabler of effective rescue execution, allowing multiple agents to make simultaneous progress instead of interfering or idling.

Coverage alone is insufficient without coordination. Coverage alone is insufficient without effective coordination like MOSAIC’s. Although several methods exhibit steady increases in coverage over time, high coverage does not necessarily translate into high transport rates. In particular, planners without structured memory or with weak coordination mechanisms often achieve moderate-to-high coverage but remain limited in transport performance due to poor agent balance and inefficient task allocation.

In contrast, MOSAIC-augmented methods consistently convert coverage into successful transport by improving coordination

Table 4. Scalability analysis of MOSAIC across AI2-THOR for multiple agent counts. Subtables show (a) Success Rate, (b) Failure Rate, and (c) Runtime, demonstrating improvements in task performance and efficiency when MOSAIC is integrated with baseline planners.

(a) Success Rate as a function of Agent Count.						(b) Failure Rate as a function of Agent Count.					
Agent Count →	1	2	3	4	5	Agent Count →	1	2	3	4	5
ACT	0.19	0.42	0.55	0.46	0.47	ACT	0.55	0.42	0.38	0.44	0.45
CoT	0.02	0.08	0.14	0.13	0.13	CoT	0.52	0.47	0.45	0.48	0.48
REACT	0.23	0.44	0.57	0.50	0.48	REACT	0.48	0.39	0.35	0.38	0.40
REACT+MOSAIC	0.38	0.51	0.63	0.64	0.67	REACT+MOSAIC	0.42	0.36	0.28	0.25	0.22
SMARTLLM	0.01	0.06	0.13	0.12	0.10	SMARTLLM	0.58	0.51	0.48	0.50	0.52
CoELA	0.02	0.11	0.18	0.14	0.12	CoELA	0.54	0.46	0.42	0.45	0.47
LLaMAR	0.31	0.59	0.68	0.65	0.65	LLaMAR	0.40	0.32	0.25	0.28	0.28
LLaMAR+MOSAIC	0.45	0.70	0.79	0.76	0.76	LLaMAR+MOSAIC	0.28	0.20	0.12	0.10	0.08

(c) Runtime as a function of Agent Count.					
Actor Count →	1	2	3	4	5
ACT	326.2	398.1	286.5	316.4	320.9
CoT	196.1	229.1	156.8	176.1	180.3
REACT	319.6	367.5	312.5	324.1	320.6
REACT+MOSAIC	234.5	342.0	275.9	241.7	238.4
SMARTLLM	984.8	935.9	736.3	783.3	803.9
CoELA	923.3	889.4	671.1	683.5	690.6
LLaMAR	772.2	757.9	584.5	612.1	622.6
LLaMAR + MOSAIC	538.6	554.1	424.9	448.7	455.1

once relevant regions are discovered. By enabling agents to share and reuse semantic and spatial context, MOSAIC promotes balanced engagement with rescue targets and reduces redundant exploration. This demonstrates that in AI2-THOR and SAR settings, effective coordination (rather than sheer exploration breadth) is the factor that determines whether discovered information leads to successful task completion.

D.3. Generality across Models

Figure 6 reports success rate and runtime across three foundation models (GPT-4o, Claude Sonnet 4.5, and Gemini 3 Flash) under different planning methods. We analyze whether MOSAIC yields consistent improvements across models with varying success and inference cost.

MOSAIC is generalizable across foundation models. Across all three foundation models, MOSAIC consistently improves success rate when paired with both REACT and LLaMAR, indicating strong generality. For REACT, MOSAIC increases success rate by +7% points on GPT-4o (0.44 → 0.51), +4% points on Claude Sonnet 4.5 (0.45 → 0.49), and +5% points on Gemini 3 Flash (0.40 → 0.45).

When applied to LLaMAR, MOSAIC yields larger gains: +11% points on GPT-4o (0.59 → 0.70), +11% points on Claude Sonnet 4.5 (0.57 → 0.68), and +10% points on Gemini 3 Flash (0.54 → 0.64). Importantly, these improvements are achieved while reducing runtime in all cases (Figure 6b); for example, LLaMAR + MOSAIC reduces runtime by 26.9% on GPT-4o (757.9s → 554.1s), 27.1% on Claude Sonnet 4.5 (772s → 563s), and 27.5% on Gemini 3 Flash (803s → 582s). These consistent trends suggest that MOSAIC acts as a planner-agnostic augmentation that generalizes across diverse foundation models.

MOSAIC is robust even with weaker models. While absolute performance degrades as we move from GPT-4o and Claude Sonnet 4.5 to Gemini 3 Flash, the relative benefits of MOSAIC remain stable, demonstrating robustness under weaker models for the tasks of AI2THOR and SAR. For instance, baseline REACT success drops by 4% points from GPT-4o to Gemini 3 Flash (0.44 → 0.40), yet MOSAIC preserves comparable relative gains (+7% points on GPT-4o vs. +5% points

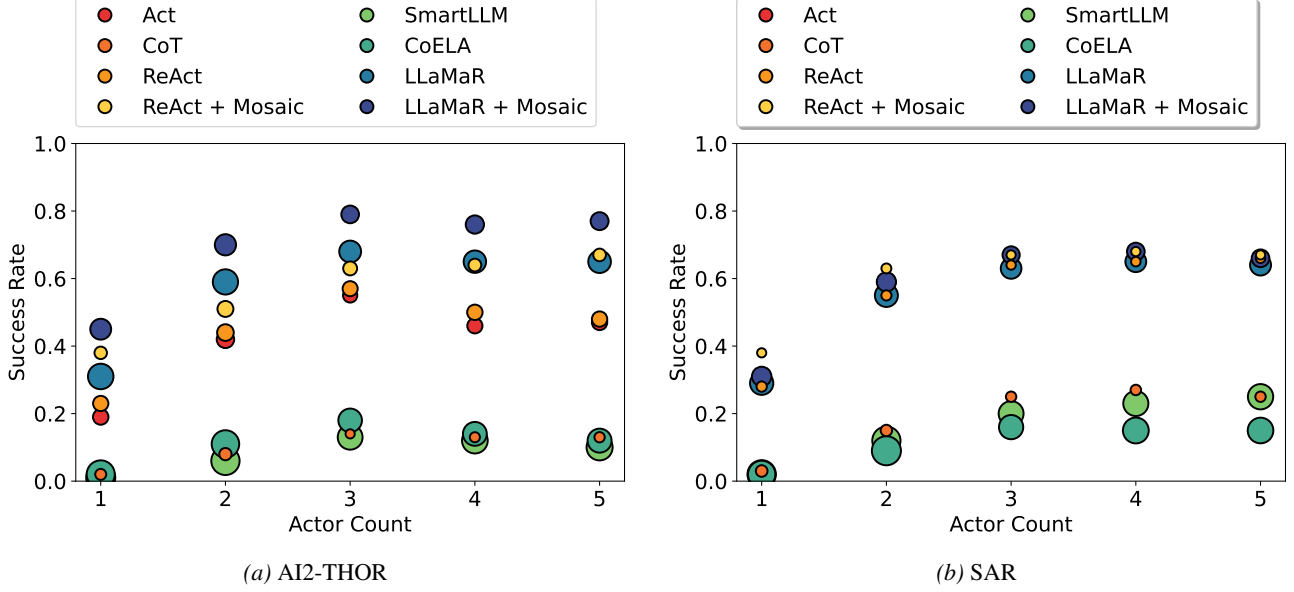


Figure 3. A three-way comparison among success rate (y-axis), actor count (x-axis), and runtime (circle size) for MOSAIC and its baselines. Smaller circles indicate lower runtime. Across actor counts, our method consistently achieves the highest success rate while incurring the lowest runtime among approaches that rely on multiple LLM calls. Moreover, increasing the number of actors yields diminishing returns, reflecting saturation in the number of actors required by the environment.

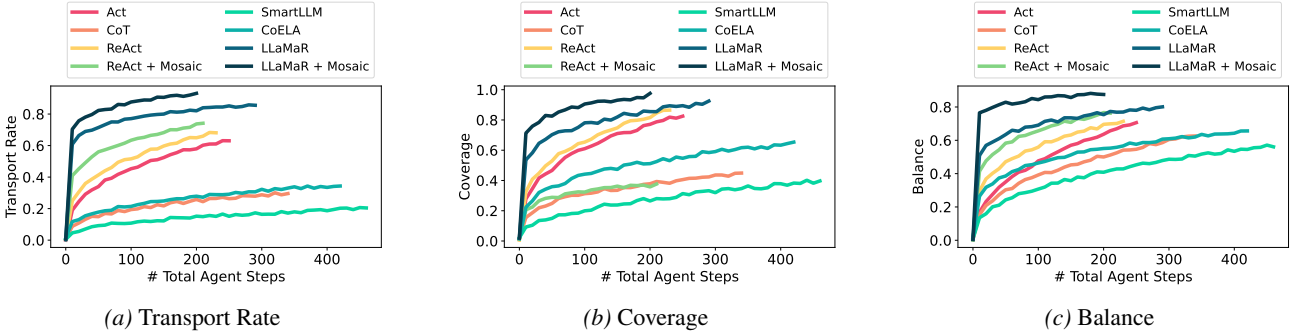


Figure 4. Transport Rate, Coverage, and Balance vs. Number of Agent Steps in AI2-THOR.

Gemini 3 Flash) against the base planners. A similar pattern holds for LLAMAR, where baseline success decreases by 5% points ($0.59 \rightarrow 0.54$), but MOSAIC maintains nearly identical absolute improvements (+11% points on GPT4o vs. +10% points on Gemini 3 Flash). Moreover, the ordering of methods remains invariant across all models, $\text{REACT} < \text{REACT} + \text{MOSAIC} < \text{LLAMAR} < \text{LLAMAR} + \text{MOSAIC}$, indicating that MOSAIC’s effectiveness is not tied to a specific foundation model’s capability. These results suggest that MOSAIC improves planning robustness by mitigating execution failures and inefficiencies, rather than relying on increased model capacity.

D.4. Performance under Observation Noise

To evaluate robustness under realistic sensing imperfections, we define three noise regimes that progressively increase localization and perception errors. Low, medium, and high noise settings apply Gaussian position noise (0.1/0.3/0.5 grid units), rotation noise ($5^\circ/10^\circ/15^\circ$), and perception errors in terms of miss rate (visible objects not detected), false positive rate (non-existent objects hallucinated), and misclassification rate (objects detected with incorrect class), set to (2%, 1%, 1%), (5%, 2%, 3%), and (10%, 5%, 5%) respectively. All regimes additionally include distance measurement errors of 5%, 10%, and 15%, enabling a controlled evaluation of MOSAIC under increasingly adverse sensing conditions.

Figure 7 summarizes success metrics under increasing noise with LLAMAR + MOSAIC, while Table 5 reports the complete

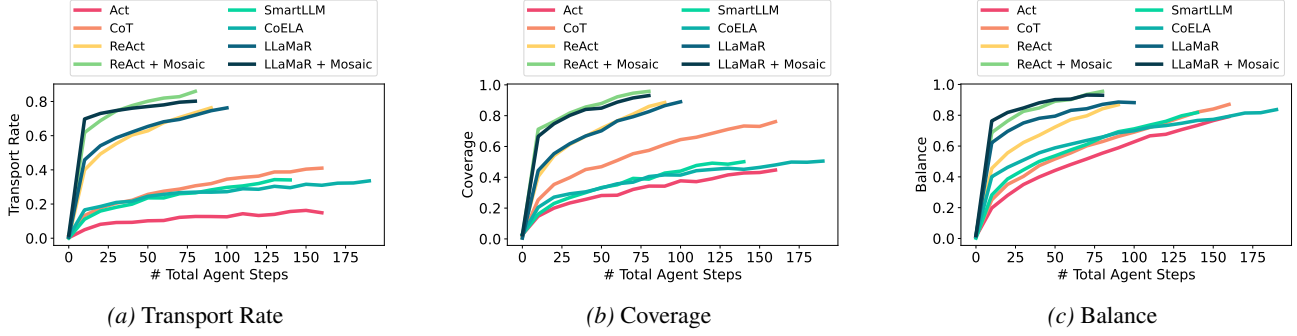


Figure 5. Transport Rate, Coverage, and Balance vs. Number of Agent Steps in SAR.

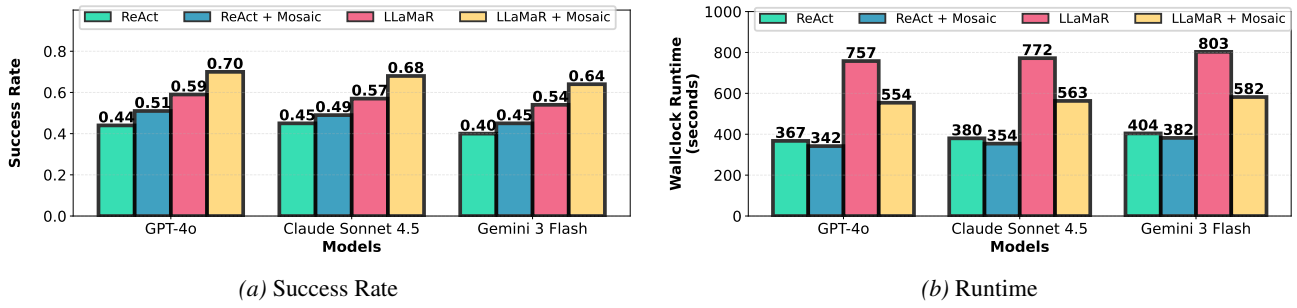


Figure 6. Evaluation of MOSAIC in AI2-THOR environment across three foundation models in terms of (a) success rate and (b) runtime. Across all models and planning methods REACT, LLAMAR, MOSAIC consistently improves task success while reducing runtime, demonstrating that its gains are model-agnostic and not tied to absolute model capability.

set of success and efficiency metrics, providing a holistic view of performance and coordination cost.

Increased noise triggers adaptive coordination and recovery behavior. As environmental noise increases, MOSAIC-based execution responds by allocating additional coordination and reasoning effort, as reflected across all efficiency metrics. From no noise to high noise, runtime increases by 88% (554s $\rightarrow$ 1042s), alongside an 83% increase in LLM calls (231 $\rightarrow$ 423), a 90% increase in total agent steps (204 $\rightarrow$ 388), and a 77% increase in total token usage (1.36M $\rightarrow$ 2.41M). Importantly, this additional coordination effort grows faster than the degradation in success rate (-54%, 0.70 $\rightarrow$ 0.32), indicating that MOSAIC actively engages in coordination to preserve task execution under uncertainty rather than prematurely terminating or collapsing. This behavior suggests that higher noise elicits proactive synchronization and resilience mechanisms, allowing agents to continue coordinated execution despite increasingly unreliable dynamics.

MOSAIC degrades gracefully under increasing noise. Through increased coordination effort, MOSAIC exhibits smooth performance degradation rather than catastrophic collapse. Success rate decreases monotonically from 0.70 (no noise) to 0.64 (low), 0.51 (medium), and 0.32 (high), while intermediate task-quality metrics remain comparatively robust. Even under high noise, MOSAIC maintains a transport rate of 0.55, coverage of 0.68, and balance of 0.59, compared to 0.93, 0.95, and 0.87 in the noiseless setting. Meanwhile, failure rate increases gradually from 20.1% $\rightarrow$ 26.2% $\rightarrow$ 42.3% $\rightarrow$ 63.7%, indicating a progressive shift toward coordination breakdowns rather than abrupt loss of task engagement. These results suggest that MOSAIC preserves partial task structure and agent contribution under noise, with failures arising primarily in late-stage coordination.

D.5. Ablation Studies

We analyze how ILP constraints and cost minimization contribute to performance, study the sensitivity of cost-function hyperparameters, and examine the runtime dynamics of the cost function.

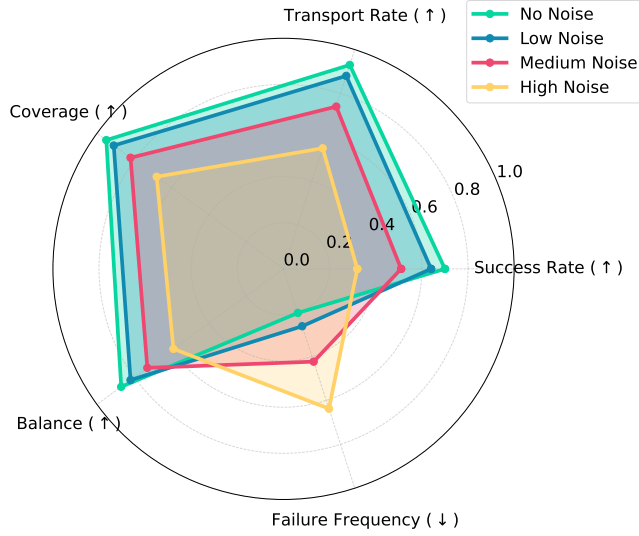


Figure 7. Key performance metrics under progressively increasing noise levels for AI2-THOR environment. As noise intensifies, the system dynamically adapts by leveraging additional LLM calls and agent steps, enabling it to maintain functionality even in the high noise settings.

Table 5. Robustness of LLAMAR + MOSAIC under increasing sensor noise in AI2-THOR with 2 agents. As sensing noise increases, MOSAIC degrades gracefully across all success metrics while adaptively increasing coordination effort, reflected in higher runtime, LLM calls, and agent steps.

LLAMAR + MOSAIC	Success Rate (↑)	Transport Rate (↑)	Coverage (↑)	Balance (↑)	Failure Rate (↓)	Runtime (Seconds, ↓)	# of LLM Calls (↓)	# of Agent Steps (↓)	# of Tokens ($\times 10^3$, ↓)
No Noise	0.70	0.93	0.95	0.87	0.20	554.1	231.5	1364	204.1
Low Noise	0.64	0.88	0.91	0.82	0.26	612.5	258.1	1492	238.6
Medium Noise	0.51	0.74	0.82	0.73	0.42	745.3	314.9	1825	295.6
High Noise	0.32	0.55	0.68	0.59	0.64	1042.1	422.6	2410	388.1

D.5.1. ROLE OF CONSTRAINTS AND COST FUNCTION IN ILP

Table 2 shows that in the single-call setting, REACT+ASM and REACT+ILP achieve identical failure rates of 0.37 on AI2-THOR, yet differ in runtime (371s vs. 356s) and agent steps (230 vs. 211). This gap arises because the ILP framework improves efficiency beyond failure avoidance by optimizing action assignment and penalizing redundant behaviors, resulting in shorter execution trajectories even when failure rates are comparable. These effects are amplified in the multi-call setting. For LLAMAR, adding ASM reduces the failure rate by 3% points, yielding runtime reduction of 4.5% and agent steps reduction of 3.9%. In contrast, adding ILP further lowers the failure rate by 8% points and produces substantially larger efficiency improvements, reducing runtime by 17.1% and reducing agent steps by 18.8%. This demonstrates that coordinated action assignment and runtime-aware optimization are critical for translating improved feasibility into significant efficiency gains, particularly in multi-call planners due to the task decomposition.

To understand the effect of ILP further, Table 6 presents an ablation study on AI2-THOR that isolates the contributions of individual components in MOSAIC relative to the strongest baseline, LLAMAR. Starting from LLAMAR, this table focuses on isolating the contributions of the ILP components by adding (i) the *Feasibility Checker (FC)* only and (ii) *Cost Function Minimization (CFM)* only. For completeness and ease of comparison, we also report results for +ASM, full +ILP (FC + CFM), and the complete MOSAIC (ASM + ILP), which are analyzed in more detail in a separate table.

To construct the ablations, FC-only enforces all feasibility and coordination constraints but selects arbitrarily among feasible joint actions without optimization, while CFM-only optimizes the cost function but does not enforce hard feasibility constraints beyond basic action validity. The full ILP combines both FC and CFM, while MOSAIC further augments ILP with semantic memory.

Feasibility constraints primarily improve coordination and balance. Adding FC alone yields consistent gains in

Table 6. Comparative evaluation of various components of MOSAIC against best -performing baseline method LLAMAR across key effectiveness and efficiency metrics in AI2-THOR environment. MOSAIC integrates *Agent-centric Semantic Memory (ASM)* with an *Integer Linear Programming (ILP)* framework for coordinated multi-agent planning. Within the ILP, the *Feasibility Checker (FC)* enforces action validity and coordination constraints, while *Cost Function Minimization (CFM)* optimizes among feasible joint actions. Together, FC and CFM constitute the ILP module that regulates and optimizes multi-agent action selection. Arrows ($\uparrow/\downarrow$) indicate whether higher or lower values are better. The numbers in subscript indicate the change relative to LLAMAR. For the first five columns (effectiveness metrics), subscript is the absolute difference, while for the last four columns (efficiency metrics) subscript is percent difference. Positive improvements are highlighted in **green**, and drops are highlighted in **red**. Best results are highlighted in bold.

Method	Effectiveness Metrics					Efficiency Metrics			
	Success Rate ($\uparrow$)	Transport Rate ($\uparrow$)	Coverage ($\uparrow$)	Balance ($\uparrow$)	Failure Rate ($\downarrow$)	Runtime (Seconds, $\downarrow$)	# of LLM Calls ($\downarrow$)	# of Agent Steps ($\downarrow$)	# of Tokens ($\times 10^3$, $\downarrow$)
LLAMAR	0.59	0.85	0.90	0.79	0.32	757.9	346.8	295.8	1472
$\hookrightarrow$ + ASM	0.61(+0.02)	0.87(+0.02)	0.91(+0.01)	0.82(+0.03)	0.29(-0.03)	723.8(- 4.5%)	310.2(-10.6%)	284.2(- 3.9%)	1562(+6.1%)
$\hookrightarrow$ + FC	0.63(+0.04)	0.88(+0.03)	0.91(+0.01)	0.84(+0.05)	0.29(-0.03)	702.0(- 7.4%)	285.3(-17.7%)	276.4(- 6.6%)	1475(+0.2%)
$\hookrightarrow$ + CFM	0.65(+0.06)	0.89(+0.04)	0.92(+0.02)	0.84(+0.05)	0.26(-0.06)	665.1(-12.2%)	271.8(-21.6%)	258.3(-12.7%)	1447(-1.7%)
$\hookrightarrow$ + ILP	0.67(+0.08)	0.90(+0.05)	0.93(+0.03)	0.85(+0.06)	0.24(-0.08)	628.1(-17.1%)	258.4(-25.5%)	240.3(-18.8%)	1420(-3.5%)
$\hookrightarrow$ + MOSAIC	0.69 (+0.10)	0.92 (+0.07)	0.95 (+0.05)	0.87 (+0.08)	0.20 (-0.12)	554.1 (-26.9%)	231.5 (-33.3%)	204.1 (-31.0%)	1364 (-7.3%)

coordination-related metrics, highlighting the importance of enforcing joint-action feasibility. Relative to LLAMAR, FC improves success and transport rates by 4% and 3% points, respectively, while increasing balance by 5% points and reducing failure rate by 3% points. These gains translate into efficiency improvements, including a 17.7% reduction in LLM calls and a 6.6% reduction in agent steps, despite minimal change in token usage (+0.2%). This indicates that enforcing feasibility constraints alone already prevents conflicting or redundant actions, leading to better workload distribution and fewer coordination-induced failures.

Cost-aware optimization drives efficiency and amplifies effectiveness. CFM-only further improves both effectiveness and efficiency by explicitly favoring low-cost joint actions among feasible candidates. Compared to LLAMAR, CFM achieves larger gains than FC in success (6% points), transport (4% points), and coverage (2% points), while reducing failure rate by 6% points. Efficiency improvements are also more pronounced, with 21.6% fewer LLM calls, 12.7% fewer agent steps, and a 12.2% reduction in runtime. These results show that cost-aware selection meaningfully differentiates among feasible joint actions, suppressing inefficient behaviors such as unnecessary navigation or repeated failed attempts, which feasibility constraints alone cannot distinguish.

ILP and ASM deliver complementary benefits. Combining FC and CFM into the full ILP further amplifies these trends, yielding up to 8% points in success and 17.1% runtime reduction. Finally, integrating ASM (MOSAIC) produces the strongest gains across all metrics, including a 10% point increase in success, 7% point increase in transport, and a 26.9% runtime reduction, while also lowering token usage by 7.3%.

Together, these results show that FC ensures valid coordination, CFM optimizes efficiency within the feasible space, and ASM improves global context, with each component contributing distinct and complementary benefits to multi-agent planning performance.

D.5.2. SPATIAL AND TEMPORAL PENALTIES AND THEIR HYPERPARAMETER SENSITIVITY

Table 7 details the spatial and temporal inefficiency patterns used in the penalty term of Equation 2, along with their corresponding penalty definitions and weights. These patterns formalize common failure modes such as repetition, oscillation, regression, and stagnation, enabling structured penalization of inefficient behavior during planning.

Table 7. Spatial and temporal inefficiency patterns used in the penalty component of the ILP cost function (Equation 2). Each pattern captures a type of redundant or inefficient behavior, and is weighted by a penalty λ_p in Equation 2.

Pattern p	Description	Penalty $_p$	λ_p
CYCLIC	Agent repeats a sequence of length ℓ , n_{rep} times	$\ell \cdot n_{\text{rep}}$	2
FAILURE	Consecutive unsuccessful attempts of the same action	n_{fail}	2
OSCILLATION	Back-and-forth movement between locations	n_{osc}	1
BACKTRACKING	Spatial regressions or returning to previous positions	n_{back}	1
STAGNATION	Consecutive idle or no-progress steps	n_{idle}	2

Table 8. Ablation study on the ILP cost function hyperparameters (λ_t) corresponding to different spatial-temporal inefficiency patterns (Table 7): Cyclic, Failure, Oscillation, Backtracking, and Stagnation. Each planning configuration explores a different weighting strategy, illustrating trade-offs between task success rate and runtime in the AI2-THOR environment. The **Efficiency-Balanced** configuration serves as the baseline, achieving the best overall trade-off between effectiveness and efficiency.

Planning Configuration	Hyperparameters	Success Rate ($\uparrow$)	Runtime (s, $\downarrow$)
Efficiency-balanced	(2, 2, 1, 1, 2)	0.69	554.1
Aggressive temporal suppression	(3, 3, 2, 1, 1)	0.65	575.8
Spatial inefficiency-aware	(2, 2, 2, 2, 2)	0.59	726.4
Load-tolerant	(1, 1, 1, 1, 1)	0.54	751.9
Exploration-sensitive	(2, 2, 3, 1, 1)	0.60	720.3
Failure-dominant	(2, 4, 2, 1, 1)	0.67	550.8

Building on these pattern definitions, Table 8 presents an ablation study on the ILP cost function hyperparameters, highlighting how different weighting strategies for spatial-temporal inefficiency patterns affect planning performance. The **Efficiency-Balanced configuration achieves the highest success rate** while maintaining low runtime, demonstrating that a moderate, balanced weighting effectively filters redundant actions without over-constraining agents. Configurations that heavily prioritize temporal penalties, such as Aggressive Temporal Suppression, reduce early inefficiencies but slightly increase runtime due to load imbalance, whereas Spatial Inefficiency-Aware and Load-Tolerant settings emphasize exploration or fairness at the cost of efficiency. The Failure-Dominant variant shows that emphasizing execution failures can match baseline success while slightly improving runtime. Overall, these results indicate that careful tuning of pattern-level penalties enables the ILP to balance feasibility, efficiency, and workload distribution, reinforcing the robustness of the proposed formulation.

D.5.3. COST FUNCTION DYNAMICS

Figure 8 illustrates the evolution of the spatial-temporal penalty, load penalty, and their weighted combination over planning steps; we analyze these trends to understand how the ILP cost function formulation (Equation 1) shapes coordination behavior over time.

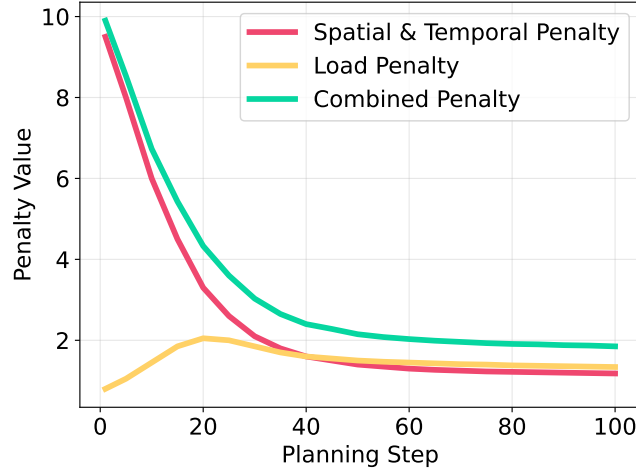


Figure 8. Evolution of ILP cost components over planning steps. The spatial-temporal penalty decreases sharply early on, reflecting rapid elimination of inefficient actions, while the load penalty rises and falls, signaling emergent workload imbalance. ILP cost function effectively prunes unproductive behaviors while promoting stable and balanced multi-agent coordination under partial observability of AI2-THOR.

Early cost reduction is dominated by the removal of inefficient actions. The combined ILP cost decreases sharply in the early planning stage, dropping from 9.90 at step #1 to 3.03 by step #30, accounting for over 65% of the total reduction observed by step #100. This rapid decrease is primarily driven by the spatial-temporal penalty, which falls from 9.5 to 2.1 over the same interval. Such behavior indicates that the cost function is most effective at filtering out clearly inefficient

joint actions (such as repeated failures, oscillations, and backtracking) early in the planning. The cost function formulation quickly prunes unproductive behaviors, which directly explains the substantial reductions in agent steps and execution failures observed in Table 2. This early pruning effect highlights the cost function’s role as a proactive action regulator rather than a purely reactive correction mechanism.

Load imbalance emerges as a meaningful coordination signal rather than an initial error. In contrast to the spatial-temporal penalty, the load penalty exhibits a non-monotonic trajectory, increasing from 0.80 at step #1 to a peak of 2.05 at step #19 before gradually decreasing to 1.34 by step #100. The delayed increase and gradual correction of the load penalty demonstrate that imbalance is treated as an emergent signal that becomes meaningful only after sufficient execution history is available. By avoiding premature penalization, the ILP allows necessary action assignments early on while still promoting balanced utilization over longer horizons, consistent with the steady improvements in balance metrics reported in Table 2.

Smooth late-stage decay indicates stable and well-regularized coordination. After approximately 40 planning steps, all cost components exhibit a slow and smooth decay, with the combined cost decreasing modestly from 2.40 at step #40 to 1.85 at step #100. This behavior suggests that the history-based penalties do not induce overcorrection or instability, but instead act as a stabilizing regularizer once agents converge to productive behaviors. Importantly, the cost converges to a low but non-zero value, capturing unavoidable coordination overhead under partial observability and shared resources, rather than indicating failure to optimize. The stability of the late-stage cost aligns with the observed gains in success and coverage, indicating that efficiency improvements do not come at the expense of task completion.

D.5.4. HYPERPARAMETER SELECTION FOR ACTION CANDIDATES

Table 9 analyze the impact of action candidate selection on planning effectiveness and efficiency. The results show that using three action candidates per agent consistently achieves the best trade-off, maximizing task success, coverage, and coordination balance while substantially reducing runtime and LLM overhead. Increasing the candidate count beyond this point yields diminishing returns, with negligible effectiveness gains but no further efficiency improvements. Moreover, allocating candidates per agent significantly outperforms an equivalent global candidate budget, as shown in Table 10, highlighting that per-agent candidates suggest higher quality actions based on each agent’s local context.

Table 9. Effect of action candidate count per agent on effectiveness and efficiency in the AI2-THOR environment using LLAMA + MOSAIC. Increasing the candidate set improves both task success and coordination up to 3 candidates per agent, which emerges as the sweet spot. With 3 action candidates per agent, we achieve the best overall effectiveness while substantially reducing runtime, LLM calls, agent steps, and token usage. Further increases yield diminishing returns, offering marginal gains in effectiveness with no meaningful efficiency improvements.

Action Candidate per Agent	Effectiveness Metrics					Efficiency Metrics			
	Success Rate (↑)	Transport Rate (↑)	Coverage (↑)	Balance (↑)	Failure Rate (↓)	Runtime (Seconds, ↓)	# of LLM Calls (↓)	# of Agent Steps (↓)	# of Tokens ($\times 10^3$, ↓)
1	0.62	0.84	0.86	0.78	0.29	705.4	284.2	275.2	1556
2	0.64	0.87	0.89	0.84	0.24	657.2	267.7	286.9	1425
3	0.69	0.92	0.95	0.87	0.20	554.1	231.5	204.1	1364
4	0.68	0.91	0.92	0.88	0.21	551.6	234.9	211.2	1355
5	0.67	0.91	0.94	0.85	0.20	552.5	229.3	201.5	1339

Table 10. Comparison of per-agent versus global action candidate selection in the AI2-THOR environment using LLAMA + MOSAIC. Allocating 3 action candidates per agent significantly outperforms an equivalent global budget of nine candidates in both effectiveness and efficiency. This highlights the importance of per-agent candidate allocation over a larger unstructured candidate pool which doesn’t take into account the agent’s local context.

Action Candidate per Agent	Effectiveness Metrics					Efficiency Metrics			
	Success Rate (↑)	Transport Rate (↑)	Coverage (↑)	Balance (↑)	Failure Rate (↓)	Runtime (Seconds, ↓)	# of LLM Calls (↓)	# of Agent Steps (↓)	# of Tokens ($\times 10^3$, ↓)
Per-Agent Candidates (3 per agent)	0.69	0.92	0.95	0.87	0.20	554.1	231.5	204.1	1364
Global Candidate Pool (9 total)	0.54	0.72	0.76	0.71	0.36	898.7	394.6	384.2	1787

D.6. Planning Step Budget

Table 11 summarizes the trade-offs between effectiveness and efficiency as the planning step budget increases.

A planning budget of 100 steps emerges as a sweet spot, offering strong effectiveness gains without disproportionate

efficiency costs. Across all methods, increasing the budget from 50 to 100 steps leads to substantial improvements in success rate by +0.21–0.25, transport rate by +0.20–0.26, coverage by +0.09–0.15, and balance by +0.14–0.17, alongside a notable reduction in failure rate of -0.13–0.19. These gains indicate that 100 steps provide sufficient planning horizon for resolving most coordination and execution challenges. Beyond this point, improvements become marginal: moving from 100 to 150 or 200 steps yields only incremental effectiveness gains (i.e., success rate improves only by 0.04–0.05% points from 100 to 150 planning steps and only by 0.01–0.02% points from 150 to 200 planning steps), suggesting diminishing returns.

Table 11. Performance metrics for various planning step budgets. While increasing the planning step limit initially boosts success rates and environmental coverage, the gains show diminishing returns and plateauing efficiency beyond a budget of 100 steps.

Planning Step Budget	2 Agent	Effectiveness Metrics					Efficiency Metrics			
		Success Rate (↑)	Transport Rate (↑)	Coverage (↑)	Balance (↑)	Failure Rate (↓)	Runtime (Seconds, ↓)	# of LLM Calls (↓)	# of Agent Steps (↓)	# of Tokens ($\times 10^3$, ↓)
50	ReAct	0.23	0.42	0.71	0.54	0.58	235.9	85.0	150.3	412.9
	↪ + MOSAIC	0.30	0.51	0.76	0.59	0.52	222.3	79.1	140.5	470.6
	LLaMaR	0.35	0.61	0.81	0.65	0.45	492.6	225.4	192.3	956.8
	↪ + MOSAIC	0.44	0.72	0.86	0.72	0.38	360.2	150.5	132.7	886.6
100	ReAct	0.44	0.68	0.86	0.71	0.39	367.5	135.4	235.4	649.6
	↪ + MOSAIC	0.51	0.73	0.90	0.76	0.36	342.2	121.7	216.1	724.8
	LLaMaR	0.59	0.85	0.92	0.79	0.32	757.9	346.8	295.8	1472.2
	↪ + MOSAIC	0.69	0.92	0.95	0.87	0.20	554.1	231.5	204.1	1364.0
150	ReAct	0.48	0.72	0.89	0.75	0.35	416.9	166.2	254.2	700.9
	↪ + MOSAIC	0.56	0.78	0.92	0.81	0.32	389.4	161.4	233.4	781.9
	LLaMaR	0.64	0.92	0.94	0.83	0.28	885.8	364.1	310.6	1545.6
	↪ + MOSAIC	0.73	0.96	0.97	0.91	0.17	626.8	243.1	214.3	1432.2
200	ReAct	0.49	0.74	0.91	0.77	0.34	511.6	201.6	303.6	786.9
	↪ + MOSAIC	0.58	0.81	0.94	0.82	0.32	483.8	196.3	282.6	830.9
	LLaMaR	0.66	0.90	0.95	0.85	0.26	901.2	401.1	346.5	1485.3
	↪ + MOSAIC	0.74	0.97	0.99	0.93	0.16	682.9	297.7	288.4	1329.5

Efficiency metrics further reinforce 100 steps as the most balanced choice. While higher budgets consistently increase runtime, LLM calls, agent steps, and token usage, the jump from 50 to 100 steps incurs a moderate runtime overhead of $\approx 53\%$ relative to the substantial effectiveness gains. In contrast, budgets of 150 and 200 steps reduce efficiency (by increasing the runtime by 4–13% and 9–24% for 100→150 and 100→200, respectively) without commensurate improvements in success or decrease in the failure rate. This plateauing behavior highlights that larger budgets primarily amplify cost rather than performance.

D.7. Standard Deviation and Variance

Tables 12 and 13 report the standard deviation of effectiveness and variance of efficiency metrics across AI2-THOR and SAR tasks, highlighting the stability of different planning components and their combinations.

The reported variance is minimal relative to the substantial efficiency gains achieved. For example, in AI2-THOR, LLAMAR averages 757 ± 14.44 s while LLAMAR+MOSAIC averages 554.1 ± 9.13 s. Given the lack of overlap between these confidence intervals, the 203s average reduction of 2 is robust. Furthermore, the variance reflects the inherent diversity across 180 distinct test cases (5 floorplans $\times$ 36 tasks) of AI2-THOR, ranging from short to complex scenarios, rather than inconsistent performance.

D.8. Library and Hardware

The MOSAIC framework integrates several core libraries. **AI2-THOR** (v5.0.0), released under the Apache 2.0 License, provides the 3D photorealistic household simulation with agent control and object interaction support. **OpenCV** (v4.9.0), distributed under the Apache 2.0 License, is used for frame capture, image processing, and video recording, while **Open3D**, released under the MIT License, handles 3D point cloud operations for spatial visualization. The ILP is solved using the CP-SAT backend from **OR-Tools** (v9.14.6206), which is also distributed under the Apache 2.0 License. All experiments are executed on a system equipped with an **AMD Ryzen 7 CPU** (8 cores, 16 threads) and **16GB RAM**.

Table 12. Standard deviation of effectiveness metrics and variance of efficiency metrics for AI2-THOR environment Table 2. MOSAIC integrates *Agent-centric Semantic Memory (ASM)* and an *Integer Linear Programming (ILP)* framework for coordinated multi-agent planning.

Method Class	Method	Effectiveness Metrics (Standard Deviation)					Efficiency Metrics (Variance)			
		Success Rate	Transport Rate	Coverage	Balance	Failure Rate	Runtime (Seconds)	# of LLM Calls	# of Agent Steps	# of Tokens ($\times 10^3$)
Single LLM call per planning step	ACT	0.04	0.03	0.04	0.04	0.06	159.73	81.29	57.16	107
	CoT	0.03	0.05	0.07	0.02	0.05	113.61	39.85	56.25	73
	REACT	0.06	0.05	0.06	0.04	0.05	155.32	53.2	65.2	80
	$\hookrightarrow$ + ASM	0.03	0.04	0.05	0.03	0.05	141.82	75.46	54.09	118
	$\hookrightarrow$ + ILP	0.04	0.04	0.04	0.03	0.04	132.57	71.38	50.11	115
	$\hookrightarrow$ + MOSAIC	0.03	0.03	0.03	0.02	0.03	96.44	69.25	42.18	121
Multiple LLM calls per planning step	SMARTLLM	0.02	0.04	0.06	0.04	0.07	239.08	27.52	102.38	83
	CoELA	0.03	0.05	0.05	0.05	0.06	264.75	76.49	131.07	155
	LLAMAR	0.06	0.05	0.04	0.02	0.04	208.51	87.23	47.23	145
	$\hookrightarrow$ + ASM	0.04	0.04	0.04	0.03	0.04	167.23	82.16	44.58	149
	$\hookrightarrow$ + ILP	0.04	0.04	0.03	0.03	0.04	121.67	81.04	39.77	148
	$\hookrightarrow$ + MOSAIC	0.05	0.04	0.04	0.03	0.03	83.46	80.48	36.49	153

Table 13. Standard deviation of effectiveness metrics and variance of efficiency metrics for SAR environment in Table 2. MOSAIC integrates *Agent-centric Semantic Memory (ASM)* and an *Integer Linear Programming (ILP)* framework for coordinated multi-agent planning.

Method Class	Method	Effectiveness Metrics					Efficiency Metrics			
		Success Rate	Transport Rate	Coverage	Balance	Failure Rate	Runtime (Seconds)	# of LLM Calls	# of Agent Steps	# of Tokens ($\times 10^3$)
Single LLM call per planning step	ACT	0.05	0.06	0.08	0.05	0.07	62.3	91.4	78.6	98
	CoT	0.04	0.05	0.07	0.04	0.06	59.8	86.1	74.2	91
	REACT	0.06	0.06	0.06	0.05	0.05	54.2	79.3	66.1	84
	$\hookrightarrow$ + ASM	0.04	0.05	0.05	0.04	0.04	49.7	74.6	60.8	92
	$\hookrightarrow$ + ILP	0.03	0.04	0.04	0.03	0.03	44.1	69.2	55.3	89
	$\hookrightarrow$ + MOSAIC	0.03	0.03	0.03	0.03	0.02	38.5	63.7	49.2	90
Multiple LLM calls per planning step	SMARTLLM	0.04	0.06	0.07	0.05	0.06	142.8	118.3	83.4	104
	CoELA	0.04	0.05	0.06	0.05	0.05	151.9	132.6	94.1	186
	LLAMAR	0.06	0.06	0.05	0.04	0.04	128.6	112.8	71.9	133
	$\hookrightarrow$ + ASM	0.05	0.05	0.05	0.04	0.03	112.4	104.1	67.8	129
	$\hookrightarrow$ + ILP	0.04	0.04	0.04	0.03	0.03	92.6	91.7	58.4	121
	$\hookrightarrow$ + MOSAIC	0.04	0.04	0.04	0.03	0.02	78.3	85.4	51.9	114

E. Qualitative Analysis

Tables 15–18 and 20–22 summarize execution trajectories produced by LLAMAR, the best-performing baseline, across two multi-agent tasks: Task 1, which involves turning off faucets and lights, and Task 2, which requires opening drawers while handling navigation challenges. These trajectories highlight systematic execution failures characteristic of this planner. The agents frequently exhibit *cyclic behavior*, repeatedly executing action sequences without progress; for instance, in the Faucet and Light task, *Agent B* cycles between the Microwave and Stool locations for over 11 repetitions, while in the Drawers task, similar navigation patterns recur 5–7 times per trajectory. *Consecutive failures* are also prevalent, with *Agent A* attempting the same unsuccessful action (such as toggling the Faucet) more than 20 times consecutively, and navigation failures in the Drawers task occurring in runs of 5–10 planning steps. *Oscillation* further manifests as back-and-forth movement between locations without net progress, reaching 10–15 consecutive steps in the Drawers task. In addition, *backtracking*, where agents unnecessarily return to previously visited positions, appears frequently, such as *Agent B* repeatedly navigating back to the Microwave or *Agent A* revisiting already-opened drawers. These behaviors lead to extended periods of *stagnation*, with stretches of 10–20 steps exhibiting no task progress. The failures are compounded by pronounced *load imbalance*, where one agent remains active while the other is stuck or idle; across failure trajectories, approximately 60% of productive actions are carried out by a single agent, while the other contributes less than 30%.

In contrast, Tables 14 and 19 show that MOSAIC completes both tasks rapidly and consistently, requiring only 7 steps for the Faucet and Light task and 11 steps for the Open Drawers task. These trajectories exhibit balanced agent participation and low action repetition. They also show consistent progress toward task completion, driven by a higher rate of successful actions, with no prolonged cycles, oscillations, or stagnation.

Overall, these qualitative results indicate that LLAMAR’s execution is dominated by repeated cyclic patterns, stagnation, and poor coordination, whereas MOSAIC suggests that incorporating failure awareness, explicit task allocation, and spatially grounded planning can substantially improve multi-agent efficiency and reliability.

Table 14. Task: Turn Off Faucet and Light. Execution trajectory generated by MOSAIC on LLAMAR. Actions taken by each agent are rendered as bounded color patches, where green indicates successful execution and red indicates failed execution.

Planning Step	Actions of Agent A	Actions of Agent B	Notes
1	NavigateTo(CounterTop_1)	NavigateTo(DiningTable_1)	Initial exploration to search for targets
2	Explore(Kitchen)	Explore(LivingRoom)	Agents scan their respective areas
3	NavigateTo(SinkBasin_1)	NavigateTo(Wall_1)	Moving closer to potential target locations
4	NavigateTo(Faucet_1)	Idle	Agent A locates faucet; B still searching
5	ToggleObjectOff(Faucet_1)	NavigateTo(LightSwitch_1)	A completes task; B finds light switch
6	Idle	ToggleObjectOff(LightSwitch_1)	B completes task; A waits
7	Done	Done	SUCCESS (7 Steps, balanced workload)

Table 15. **Task: Turn Off Faucet and Light.** Execution trajectory generated by LLAMAR. Actions taken by each agent are rendered as bounded color patches, where green indicates successful execution and red indicates failed execution.

Planning Step	Actions of Agent A	Actions of Agent B	Notes/Failure Patterns
1	ToggleObjectOff(Faucet_1)	NavigateTo(Microwave_1)	SPATIAL: Agent A too far
2	ToggleObjectOff(Faucet_1)	NavigateTo(Microwave_1)	FAILURE: Agent A repeats (2×)
3	ToggleObjectOff(Faucet_1)	NavigateTo(Stool_1)	FAILURE: Agent A repeats (3×)
4	ToggleObjectOff(Faucet_1)	NavigateTo(Stool_2)	SPATIAL: Agent A frozen
5	ToggleObjectOff(Faucet_1)	NavigateTo(Stool_2)	OSCILLATION: Agent B repeats
6	ToggleObjectOff(Faucet_1)	NavigateTo(Microwave_1)	BACKTRACKING: Agent B returns
7	ToggleObjectOff(Faucet_1)	NavigateTo(Stool_2)	CYCLIC: Microwave→Stool
8	ToggleObjectOff(Faucet_1)	NavigateTo(Microwave_1)	BACKTRACKING: Agent B (3×)
9	ToggleObjectOff(Faucet_1)	NavigateTo(Stool_2)	CYCLIC: Repeat of Step 7
10	ToggleObjectOff(Faucet_1)	NavigateTo(Stool_2)	OSCILLATION: Agent B (3×)
11	ToggleObjectOff(Faucet_1)	NavigateTo(Microwave_1)	BACKTRACKING: Agent B (4×)
12	ToggleObjectOff(Faucet_1)	NavigateTo(Stool_2)	CYCLIC: 3rd cycle
13	ToggleObjectOff(Faucet_1)	NavigateTo(Stool_2)	STAGNATION: 13 steps, no progress
... continued in the next table			

Table 16. **Task: Turn Off Faucet and Light (continued).** Execution trajectory generated by LLAMAR. Actions taken by each agent are rendered as bounded color patches, where green indicates successful execution and red indicates failed execution.

Planning Step	Actions of Agent A	Actions of Agent B	Notes/Failure Patterns
	... continued from the previous table		
14	ToggleObjectOff(Faucet_1)	ToggleObjectOff(Microwave_1)	WRONG TARGET: Agent B attempts Microwave
15	ToggleObjectOff(Faucet_1)	ToggleObjectOff(Microwave_1)	FAILURE: Agent B repeats wrong target (2×)
16	ToggleObjectOff(Faucet_1)	ToggleObjectOff(Microwave_1)	FAILURE: Agent B repeats wrong target (3×)
17	ToggleObjectOff(Faucet_1)	ToggleObjectOff(Microwave_1)	CYCLIC: Both agents stuck in loops
18	ToggleObjectOff(Faucet_1)	ToggleObjectOff(Microwave_1)	SPATIAL: Agent A never reaches faucet
19	ToggleObjectOff(Faucet_1)	ToggleObjectOff(Microwave_1)	STAGNATION: 19 steps, zero progress
20	ToggleObjectOff(Faucet_1)	ToggleObjectOff(Microwave_1)	FAILURE: Agent A 20 consecutive failures
21	ToggleObjectOff(Faucet_1)	ToggleObjectOff(Microwave_1)	CYCLIC: Repeating identical actions
22	ToggleObjectOff(Faucet_1)	ToggleObjectOff(Microwave_1)	WRONG TARGET: Agent B never targets LightSwitch
23	Idle	ToggleObjectOff(Faucet_1)	WRONG TARGET: Agent B attempts Agent A's target
24	ToggleObjectOff(Faucet_1)	ToggleObjectOff(Microwave_1)	CYCLIC: Pattern resumes
25	ToggleObjectOff(Faucet_1)	ToggleObjectOff(Microwave_1)	STAGNATION: 25 steps, 0% complete
26	ToggleObjectOff(Faucet_1)	ToggleObjectOff(Microwave_1)	SPATIAL: No positional progress
27	ToggleObjectOff(Faucet_1)	ToggleObjectOff(Microwave_1)	FAILURE: Agent A 27 consecutive attempts
28	ToggleObjectOff(Faucet_1)	ToggleObjectOff(Microwave_1)	WRONG TARGET: Agent B at Microwave
29	ToggleObjectOff(Faucet_1)	NavigateTo(Microwave_1)	BACKTRACKING: Agent B navigates to Microwave
30	ToggleObjectOff(Faucet_1)	NavigateTo(Stool_2)	CYCLIC: Microwave→Stool loop
	... continued in the next table		

Table 17. **Task: Turn Off Faucet and Light (continued).** Execution trajectory generated by LLAMAR. Actions taken by each agent are rendered as bounded color patches, where green indicates successful execution and red indicates failed execution.

Planning Step	Actions of Agent A	Actions of Agent B	Notes/Failure Patterns
	... continued from previous table		
50	ToggleObjectOff(Faucet_1)	ToggleObjectOff(Microwave_1)	STAGNATION: 50 steps, 0% completion
51	ToggleObjectOff(Faucet_1)	ToggleObjectOff(Microwave_1)	SPATIAL: Agent A frozen 51 steps
52	ToggleObjectOff(Faucet_1)	ToggleObjectOff(Microwave_1)	WRONG TARGET: Agent B never finds LightSwitch
53	ToggleObjectOff(Faucet_1)	ToggleObjectOff(Microwave_1)	CYCLIC: Same toggle pattern 40+ times
54	ToggleObjectOff(Faucet_1)	ToggleObjectOff(Microwave_1)	FAILURE: Agent A succeeds, Agent B still wrong
55	ToggleObjectOff(Faucet_1)	ToggleObjectOff(Microwave_1)	TERMINAL: Both agents stuck in loops
56	ToggleObjectOff(Faucet_1)	ToggleObjectOff(Microwave_1)	SPATIAL: Distance constraint violated 56×
57	ToggleObjectOff(Faucet_1)	ToggleObjectOff(Faucet_1)	WRONG TARGET: Agent B attempts Agent A's goal
58	ToggleObjectOff(Faucet_1)	ToggleObjectOff(Microwave_1)	CYCLIC: Back to Microwave toggle
59	ToggleObjectOff(Faucet_1)	ToggleObjectOff(Microwave_1)	STAGNATION: 59 steps, no progress
60	ToggleObjectOff(Faucet_1)	ToggleObjectOff(Microwave_1)	FAILURE: 60 wasted steps
...	...	...	... continued until Step 70
70	ToggleObjectOff(Faucet_1)	ToggleObjectOff(Microwave_1)	STAGNATION: 70 steps, 0% complete
	... continued in the next table		

Table 18. **Task: Turn Off Faucet and Light (continued).** Execution trajectory generated by LLAMAR. Actions taken by each agent are rendered as bounded color patches, where green indicates successful execution and red indicates failed execution.

Planning Step	Actions of Agent A	Actions of Agent B	Notes/Failure Patterns
	... continued from the previous table		
88	ToggleObjectOff(Faucet_1)	NavigateTo(Microwave_1)	BACKTRACKING: Agent B at Microwave (10th+ time)
89	ToggleObjectOff(Faucet_1)	NavigateTo(Stool_2)	CYCLIC: Microwave→Stool pattern (10th+ cycle)
90	ToggleObjectOff(Faucet_1)	NavigateTo(Stool_2)	OSCILLATION: Agent B repeats Stool (15th+ time)
91	ToggleObjectOff(Faucet_1)	ToggleObjectOff(Microwave_1)	CYCLIC: Back to wrong target toggle
92	ToggleObjectOff(Faucet_1)	NavigateTo(Microwave_1)	BACKTRACKING: Agent B returns to Microwave (11th time)
93	ToggleObjectOff(Faucet_1)	NavigateTo(Stool_2)	CYCLIC: Same pattern as Steps 7, 9, 12, 30, 89
94	ToggleObjectOff(Faucet_1)	NavigateTo(Stool_2)	OSCILLATION: Agent B stuck in Stool loop
95	ToggleObjectOff(Faucet_1)	ToggleObjectOff(Microwave_1)	WRONG TARGET: Agent B still targeting Microwave
96	ToggleObjectOff(Faucet_1)	ToggleObjectOff(Microwave_1)	SPATIAL: Agent A at (2.5,1.0) for 96 steps
97	ToggleObjectOff(Faucet_1)	ToggleObjectOff(Microwave_1)	FAILURE: Agent A 97 consecutive failures
98	ToggleObjectOff(Faucet_1)	ToggleObjectOff(Microwave_1)	STAGNATION: 98 steps, 0% task completion
99	ToggleObjectOff(Faucet_1)	ToggleObjectOff(Microwave_1)	CYCLIC: Pattern repeated 30+ times
100	ToggleObjectOff(Faucet_1)	ToggleObjectOff(Microwave_1)	TIMEOUT

Table 19. **Task: Open Drawers (with Navigation Challenges)**. Execution trajectory generated by MOSAIC on LLAMAR. Actions taken by each agent are rendered as bounded color patches, where green indicates successful execution and red indicates failed execution.

Planning Step	Actions of Agent A	Actions of Agent B	Notes/Failure Patterns
1	NavigateTo(Drawer_1)	NavigateTo(Cabinet_4)	Parallel navigation
2	OpenObject(Drawer_1)	OpenObject(Drawer_2)	Both agents productive
3	NavigateTo(Drawer_3)	OpenObject(Drawer_3)	Agent A navigates, Agent B opens
4	OpenObject(Drawer_4)	NavigateTo(Drawer_5)	Agent B’s navigation fails, Agent A opens
5	NavigateTo(Drawer_6)	Rotate(Right)	Agent B repositions
6	OpenObject(Drawer_6)	NavigateTo(Drawer_5)	Agent A opens, Agent B navigates
7	NavigateTo(Drawer_7)	OpenObject(Drawer_5)	Both agents productive
8	OpenObject(Drawer_7)	NavigateTo(Drawer_8)	Both agents productive
9	Rotate(Right)	OpenObject(Drawer_8)	Agent A repositions
10	OpenObject(Drawer_9)	Idle	Agent A finishes last drawer
11	Done	Done	SUCCESS

Table 20. **Task: Open Drawers (with Navigation Challenges)**. Execution trajectory generated by LLAMAR. Actions taken by each agent are rendered as bounded color patches, where green indicates successful execution and red indicates failed execution.

Planning Step	Actions of Agent A	Actions of Agent B	Notes/Failure Patterns
1	OpenObject(Drawer_1)	Move(Right)	1/9 drawers
2	Move(Ahead)	Move(Ahead)	Navigation
3	Move(Left)	NavigateTo(Cabinet_3)	FAILURE: Agent A’s move fails
4	Rotate(Left)	OpenObject(Drawer_4)	2/9 drawers
5	Move(Ahead)	NavigateTo(Drawer_5)	FAILURE: Agent A blocked
6	Rotate(Right)	OpenObject(Drawer_5)	3/9 drawers
7	Move(Left)	OpenObject(Drawer_3)	OSCILLATION: Agent A Left/Right
8	Move(Right)	OpenObject(Drawer_2)	4/9 drawers
... continued in the next table			

Table 21. **Task: Open Drawers (with Navigation Challenges) (continued).** Execution trajectory generated by LLAMAR. Actions taken by each agent are rendered as bounded color patches, where green indicates successful execution and red indicates failed execution.

Planning Step	Actions of Agent A	Actions of Agent B	Notes/Failure Patterns
	... continued from the previous table		
9	NavigateTo(Drawer_1)	OpenObject(Drawer_4)	BACKTRACKING: Agent A returns to Drawer_1
10	OpenObject(Drawer_1)	NavigateTo(Drawer_4)	Agent A tries to re-open Drawer_1 (already open!)
11	Move(Right)	Move(Left)	OSCILLATION: Agent B Left/Right movement
12	Move(Ahead)	Rotate(Right)	FAILURE: Agent A can't move ahead
13	Move(Right)	NavigateTo(Drawer_6)	Agent A wanders, Agent B nav fails
14	CloseObject(Drawer_1)	Move(Back)	Agent A closes drawer (wrong action!)
15	NavigateTo(Drawer_6)	Rotate(Right)	Agent B repositions
16	OpenObject(Drawer_6)	Move(Left)	5/9 drawers, Agent B fails
17	NavigateTo(Drawer_7)	Move(Ahead)	FAILURE: Both fail navigation
18	Move(Left)	Move(Back)	OSCILLATION: Both stuck
19	Move(Back)	Move(Ahead)	OSCILLATION: Opposite directions
20	Rotate(Right)	Move(Right)	Agent A rotates, Agent B fails
	... continued in the next table		

Table 22. **Task: Open Drawers (with Navigation Challenges) (continued).** Execution trajectory generated by LLAMAR. Actions taken by each agent are rendered as bounded color patches, where green indicates successful execution and red indicates failed execution.

Planning Step	Actions of Agent A	Actions of Agent B	Notes/Failure Patterns
	... continued from the previous table		
60	OpenObject(Drawer_2)	Rotate(Right)	FAILURE: Alice tries closed drawer
61	NavigateTo(Drawer_2)	NavigateTo(Drawer_6)	FAILURE: Alice nav fails
62	Move(Back)	OpenObject(Drawer_6)	FAILURE: Both fail
63	Rotate(Right)	OpenObject(Drawer_6)	FAILURE: Bob repeats
64	Move(Left)	Move(Back)	OSCILLATION: Movement loop
65	Move(Ahead)	Move(Left)	OSCILLATION: Still stuck
66	Rotate(Right)	Move(Ahead)	Alice rotates 10+ times now
67	Move(Right)	Move(Left)	OSCILLATION: Opposite directions
68	Move(Back)	Move(Ahead)	CYCLIC: Same pattern as Step 19
69	Move(Left)	Move(Left)	FAILURE: Both fail same direction
70	Move(Left)	Move(Left)	FAILURE: Repeat of Step 69
71	Move(Back)	Move(Left)	FAILURE: 71 steps, no progress
	... till end of trajectory		TIMEOUT

F. Limitations

While MOSAIC significantly improves execution efficiency and coordination, it has a few limitations.

(a) First, the agent-centric semantic memory relies on reasonably accurate pose estimates. Hence, large localization errors or severe sensor drift could degrade state tracking and propagate to action selection. Although we evaluate robustness under moderate noise, real-world deployments with highly stochastic actuation or failures may require tighter integration with low-level control. The primary safeguard against sensor drift is closed-loop perception, where real-time observations correct accumulated errors in ASM; mapping exact distances into coarse semantic tiers (e.g., “just ahead”) further absorbs minor metric noise before it affects high-level reasoning. Consistent with standard practice in simulator environments such as AI2-THOR, we assume reliable localization; addressing raw sensor drift is orthogonal to our framework and belongs at the control layer. We also evaluate spatial noise sensitivity in Appendix D.4.

(b) Second, our feasibility constraints encode physical and coordination rules, but may not capture unforeseen dynamics or social behaviors, requiring manual extension for new domains.

(c) Lastly, like all planning-based systems operating over a restricted action space, performance depends on the coverage of candidate actions. If critical actions are absent, downstream planning cannot recover them. MOSAIC partially mitigates this through grounded prompting and iterative refinement of the LLM-Actor, improving candidate quality and coverage. Since our goal is not to improve the LLM itself, we tightly couple Agent-centric Semantic Memory (ASM) with the LLM, injecting structured spatial cues (e.g., object locations and obstacle distances) to reduce spatial unawareness and increase feasible action proposals. Execution feedback is also incorporated into subsequent steps, enabling implicit refinement and alternative proposals. Nonetheless, this remains a fundamental limitation shared across planning systems: they can only select from provided candidates, and improving exploration and proposal generation remains an orthogonal direction.

G. Prompts

We enhance the action planning prompts used by the LLM-ACTOR in LLAMAR (Nayak et al., 2024) by introducing two key modifications: (a) an explicit emphasis on runtime efficiency, and (b) semantically expressed ILP constraints that guide the LLM to generate constraint-compliant candidate actions. The resulting prompt augmentations are shown below, the original base prompt is provided in LLAMAR (Nayak et al., 2024).

Prompt Enhancements for Constraint-Aware and Runtime-Efficient Planning

You are an excellent planner and robot controller who is tasked with helping `len(AGENT_NAMES)` embodied robots named `"`, `".join(AGENT_NAMES[:-1]) + f"`, and `AGENT_NAMES[-1]"` carry out a task. All `len(AGENT_NAMES)` robots have a partially observable view of the environment. Hence they have to explore around in the environment to do the task.

****YOUR TOP PRIORITY IS RUNTIME EFFICIENCY: you must complete the task in the LEAST number of steps possible. Strictly avoid actions that are redundant, unnecessary, or likely to fail.****

[Placeholder for brevity: Action List and Definitions]

Available actions include: `navigate to object`, `pick up object`, `put object on receptacle`, `open/close object`, `slice`, `toggle`, `clean`, `look up/down`, `move in direction`, `stay idle`, `Done`. [Detailed action list and parameter definitions would appear here...]

You need to suggest MULTIPLE candidate actions (3–5 actions) for each robot at the current step. The system will then use an optimization algorithm to select the best joint action set that coordinates all agents efficiently.

[Placeholder for brevity: Input Format Description]

You will receive: - Task description - Images from each agent's perspective
 - Each agent's observations (list of visible objects) - Robots' open subtasks, completed subtasks, current subtask - Robots' combined memory

[Full input format specification would appear here...]

[Detailed reasoning instructions would appear here...]

* Candidate Actions: For EACH robot, provide a list of 3-5 candidate actions (as a list of strings). These should be diverse actions that could help the robot make progress.

****PRIORITIZE RUNTIME EFFICIENCY WHEN GENERATING CANDIDATES:****

- Select actions that maximize the speed of task completion (least total steps)
- Strictly avoid failable or redundant actions
- Focus on actions that directly progress toward subtasks
- Include exploration actions only when necessary to find unknown objects
- Include wait/idle candidates only when coordination is truly needed
- Handle failures or obstacles efficiently - try alternative approaches rather than repeating failed actions

The format should be: "AGENT_NAMES[0]'s candidate actions": ["action1", "action2", "action3", ...], "AGENT_NAMES[1]'s candidate actions": [...], etc.

Important Notes:

- Robots can hold only one object at a time
- Use spatial memory and obstacles for distance checking and navigation
- Check observation list before interaction (object must be visible and close enough)
- Maintain goals during detours around obstacles
- Open containers before placing objects in them
- Close opened containers before moving away
- Avoid extraneous actions when one action is sufficient

[Additional important notes about object interactions, navigation, and coordination would appear here...]

CRITICAL CONSTRAINTS - MUST FOLLOW WHEN GENERATING CANDIDATE ACTIONS

These constraints ensure conflict-free, coordinated multi-agent actions. When generating candidate actions, ensure they follow these constraints. The ILP solver will enforce these constraints when selecting the final joint action, but generating constraint-compliant candidates improves solution quality:

****CONSTRAINT 1: Per-Agent Action Selection****

- Each agent MUST have candidate actions that include exactly ONE action per timestep (or "stay idle" or "Done")
- Generate diverse candidate actions, but ensure each candidate is a valid

single action for that agent

- This guarantees that the joint action at each step is well-defined and avoids repeated replanning or inconsistent partial assignments

****CONSTRAINT 2: Eligibility Constraints (Action Preconditions)****

When generating candidate actions, only include actions the agent can actually perform:

- ****PickupObject****: Only include if agent is NOT holding anything. If agent is already holding an object, exclude PickupObject candidates. Attempting to pick up another object while already holding one is physically infeasible.
- ****PutObject****: Only include if agent IS holding an object. If agent is holding "nothing", exclude PutObject candidates. You cannot put down an object you don't have.
- ****Object Interactions**** (OpenObject, CloseObject, SliceObject, CleanObject, ToggleObject): Only include if the target object exists and is interactable. Check the observation list to ensure the object is visible and close enough. Navigating into occupied or non-traversable regions (like walls) is not allowed.
- ****NavigateTo****: Can always be included, but ensure the object exists in the environment and the navigation path is valid.
- ****Idle/Done****: Always eligible for any agent.

****CONSTRAINT 3: Resource Exclusivity (CRITICAL - Prevents Conflicts)****

When generating candidate actions, consider that shared resources (objects, receptacles, locations) cannot be used by multiple agents simultaneously:

- ****Same Object Manipulation****: If generating candidates for multiple agents, avoid having multiple agents' candidates all targeting the same object for manipulation (pick up, open, close, slice, clean, toggle). Include diverse targets across agents. Only ONE agent can manipulate a specific object at a time.
- ****Same Receptacle****: Avoid having multiple agents' candidates all putting objects on the same receptacle simultaneously. Each receptacle can only receive one object placement action per timestep.
- ****Navigation Conflicts****: If two agents' candidates both navigate to the same object/location, this may cause conflicts and collisions. Prefer diverse navigation targets across agents.
- ****Rule****: Generate candidate actions that distribute work across different objects/receptacles to minimize conflicts. This prevents simultaneous grasps of the same object, concurrent attempts to open the same door, or contention over shared tools.

****CONSTRAINT 4: Collision and Interference Avoidance****

Some action pairs are incompatible and cannot happen simultaneously due to spatial or kinematic constraints:

- ****Conflicting Actions on Same Object****:
 - Avoid candidates where one agent opens while another closes the same object (mutually exclusive actions)
 - Avoid candidates where agents toggle the same object on/off simultaneously
 - Avoid candidates where multiple agents try to pick up the same object
- ****Spatial Conflicts****:

- Avoid candidates where multiple agents navigate to the same location/object simultaneously (risk of collision and path blocking)
- Consider path blocking when generating navigation candidates - one agent may block another's path to an object
- Avoid overlapping target locations and other mutually exclusive spatial configurations
- ****Solution****: Generate diverse candidate actions that minimize conflicts. Include idle/wait options when coordination is needed. If actions conflict, prefer assigning the action to ONE agent and having others work on different tasks.

****CONSTRAINT 5: Multi-Actor Joint-Action Requirements****

Some heavy objects require multiple agents to manipulate simultaneously:

- ****Heavy Objects**** (sofas, tables, large furniture): May require 2+ agents to move or manipulate. These subtasks cannot be completed by a single agent.
- ****Rule****: For heavy objects, either generate candidates that involve multiple agents coordinating on the same object, OR include no manipulation candidates (wait until enough agents are available). This ensures that joint subtasks are neither under-staffed (which would lead to execution failure) nor over-staffed (which would waste actions and increase runtime).
- ****Common Heavy Objects****: Sofas, Tables, Large Cabinets, Beds, Countertops
- ****Light Objects****: Books, Cups, Plates, etc. only require 1 agent

****CONSTRAINT 6: Temporal Penalties (Avoid Inefficient Patterns)****

When generating candidate actions, avoid patterns that lead to inefficiency and increase runtime:

- ****Cyclic Patterns****: Avoid candidates that repeat recent action sequences (e.g., PickupObject(Book_1) → PutObject(Sofa_1) → PickupObject(Book_1) → PutObject(Sofa_1)). These waste steps without making progress.
- ****Failure Loops****: If an action failed 2+ times consecutively, include alternative approaches in candidates instead of repeating the same failed action. Repeating failed actions wastes time.
- ****Oscillation****: Avoid candidates that create back-and-forth movement (e.g., navigating to Object_1, then Object_2, then back to Object_1). This increases total path length.
- ****Backtracking****: Avoid candidates that return to locations/objects recently visited. This wastes motion and increases runtime.
- ****Stagnation****: Avoid consecutive idle candidates when there's work to be done. Agents should be making progress when possible.

****CONSTRAINT 7: Load Balancing****

Generate candidate actions that distribute work evenly across agents:

- Include candidates that allow idle agents to help with tasks
- Avoid candidates where one agent does all the work while others are idle
- Include candidates that balance workload (e.g., if one agent is carrying objects, include candidates for other agents to help with navigation or other tasks)
- This ensures efficient parallelization and reduces the total time to complete the task

```

**BEFORE OUTPUTTING CANDIDATE ACTIONS - CONSTRAINT CHECKLIST:**
1. Each agent has 3-5 diverse candidate actions
2. No agent has PickupObject candidates if they're already holding something
3. No agent has PutObject candidates if they're holding "nothing"
4. Candidate actions distribute work across different objects (minimize same-object conflicts)
5. No conflicting candidates (e.g., one agent opening while another closes same object)
6. Heavy objects have coordination candidates OR no manipulation candidates
7. Candidates avoid cyclic patterns, failure loops, and oscillation
8. Candidates allow for load balancing across agents

* NOTE: DO NOT OUTPUT ANYTHING EXTRA OTHER THAN WHAT HAS BEEN SPECIFIED
Let's work this out in a step by step way to be sure we have the right answer.

```

We also incorporate a dedicated prompt component that instructs the LLM to reason explicitly over agent-centric spatial memory and obstacle-relative information for navigation and coordination.

Prompt related to the use of Agent-centric Spatial Memory

```

* Memory: You will receive **AGENT-CENTRIC MEMORY** and **OBSTACLES** with relative locations and distances.

**AGENT-CENTRIC MEMORY** shows object locations: "Book_1 - Alice: Far left (4 moves), Bob: Just ahead (1 move)" - **OBSTACLES** shows immediate surroundings with distances: "Ahead: Wall_1 (2 moves), Chair_1 (1 move); Left: Clear"

**Use these for all navigation decisions!**
The coordinate system is NOT meaningful - only use relative descriptions.

Your task is to record task progress and key decisions:
**DON'T repeat spatial info** (already provided). Instead, focus on:
  -What agents are HOLDING: "Alice holding Book_1, Bob holding nothing"
  -TASK PROGRESS: "COMPLETED: Laptop_1 on Sofa. REMAINING: Book_1, RemoteControl_1 need transport"
  -RECENT ACTIONS & OUTCOMES: Track what worked and what failed, adapt strategies
  -NAVIGATION STRATEGY: "Alice navigating around Chair to reach Laptop" (use OBSTACLES distances to plan)
  -MAINTAIN GOALS DURING DETOURS: Always state the ultimate goal when taking workarounds
  -CROSS-AGENT COORDINATION: Agents share discoveries and help each other navigate
**KEY PRINCIPLE**: Focus on TASK DECISIONS using provided spatial context and obstacle distances

```